

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ  
Федеральное государственное автономное образовательное учреждение высшего  
образования  
«Национальный исследовательский Нижегородский государственный университет им.  
Н.И. Лобачевского»  
(ННГУ)  
Институт информационных технологий, математики и механики

ОТЧЕТ  
по практике  
«Идентификация функции конкурентоспособности методами машинного обучения на  
основе динамики спроса»

Выполнил: студент группы  
382006-1

Мусин А.Н. \_\_\_\_\_

Проверил: доцент, к.ф.-м.н.  
Кузенков О.А. \_\_\_\_\_

Нижний Новгород  
2024

# Содержание

|       |                                                                                         |    |
|-------|-----------------------------------------------------------------------------------------|----|
| 1     | Введение . . . . .                                                                      | 5  |
| 1.1   | Актуальность и мотивация исследования . . . . .                                         | 5  |
| 1.2   | Анализ существующих решений . . . . .                                                   | 5  |
| 1.3   | Цели и задачи . . . . .                                                                 | 6  |
| 1.4   | Научная новизна . . . . .                                                               | 6  |
| 1.5   | Практическая значимость . . . . .                                                       | 7  |
| 2     | Теоретические основы . . . . .                                                          | 8  |
| 2.1   | PTX как промежуточное представление CUDA . . . . .                                      | 8  |
| 2.1.1 | Место PTX в стеке компиляции CUDA . . . . .                                             | 8  |
| 2.1.2 | Типовая система PTX . . . . .                                                           | 9  |
| 2.1.3 | Классификация инструкций PTX . . . . .                                                  | 9  |
| 2.1.4 | Директивы PTX и их роль в интерпретаторе . . . . .                                      | 9  |
| 2.1.5 | Пространства адресов и их семантика . . . . .                                           | 10 |
| 2.2   | Математическая модель SIMT-исполнения . . . . .                                         | 10 |
| 2.2.1 | Варп как алгебраическая структура . . . . .                                             | 10 |
| 2.2.2 | Маска исполнения: операции и инварианты . . . . .                                       | 11 |
| 2.2.3 | Предикатные регистры и составная маска . . . . .                                        | 11 |
| 2.3   | Математическая модель метрик производительности . . . . .                               | 11 |
| 2.3.1 | Метрика <code>branch_efficiency</code> . . . . .                                        | 11 |
| 2.3.2 | Метрика <code>bank_conflicts</code> . . . . .                                           | 12 |
| 2.3.3 | Метрики <code>global_mem_transactions</code> и <code>global_coalescing</code> . . . . . | 12 |
| 2.4   | Операционная семантика PTX (структурный подход) . . . . .                               | 13 |
| 2.4.1 | Нотация малых шагов (small-step semantics) . . . . .                                    | 13 |
| 2.4.2 | Семантика арифметических инструкций . . . . .                                           | 13 |
| 2.4.3 | Семантика инструкций сравнения и выбора . . . . .                                       | 13 |
| 2.4.4 | Семантика инструкций управления потоком . . . . .                                       | 14 |
| 2.4.5 | Семантика инструкций памяти . . . . .                                                   | 14 |
| 2.5   | Механизм динамической подстановки библиотек в Linux . . . . .                           | 14 |
| 2.5.1 | Загрузчик ELF и таблица динамических символов . . . . .                                 | 14 |
| 2.6   | Обзор смежных работ . . . . .                                                           | 15 |
| 2.6.1 | Функциональные эмуляторы: Ocelot, CUDAsim . . . . .                                     | 15 |
| 2.6.2 | Микроархитектурные симуляторы: GPGPU-Sim, Accel-Sim . . . . .                           | 15 |
| 3     | Механизм перехвата CUDA Runtime . . . . .                                               | 16 |
| 3.1   | Архитектура перехвата через LD_PRELOAD . . . . .                                        | 16 |
| 3.1.1 | Схема взаимодействия компонентов . . . . .                                              | 16 |
| 3.1.2 | Реализованные точки перехвата CUDA Runtime API . . . . .                                | 16 |
| 3.1.3 | Извлечение PTX-текста через <code>cuobjdump</code> . . . . .                            | 16 |
| 3.1.4 | Регистрация функций: отображение дескриптор $\rightarrow$ имя ядра . . . . .            | 17 |
| 3.1.5 | Запуск <code>cuemu</code> : CLI-лончер и переменные окружения . . . . .                 | 18 |
| 4     | Синтаксический анализатор PTX . . . . .                                                 | 19 |
| 4.1   | Требования к парсеру и выбор стратегии . . . . .                                        | 19 |
| 4.1.1 | Анализ структуры PTX-текста . . . . .                                                   | 19 |
| 4.1.2 | Однопроходный инкрементный разбор . . . . .                                             | 19 |
| 4.2   | Реализация парсера . . . . .                                                            | 19 |
| 4.2.1 | Разбор объявлений функций и параметров . . . . .                                        | 19 |
| 4.2.2 | Таблица меток и граф базовых блоков . . . . .                                           | 20 |

|       |                                                                                                 |    |
|-------|-------------------------------------------------------------------------------------------------|----|
| 4.2.3 | Разбор отдельной инструкции . . . . .                                                           | 20 |
| 4.2.4 | Разрешение операндов . . . . .                                                                  | 21 |
| 4.3   | Внутреннее представление инструкций . . . . .                                                   | 21 |
| 4.3.1 | Иерархия классов инструкций и монотонный счётчик PC . . . . .                                   | 21 |
| 5     | Система исполнения инструкций . . . . .                                                         | 23 |
| 5.1   | Таблица диспетчеризации . . . . .                                                               | 23 |
| 5.1.1 | Структура: фабрика и виртуальный вызов . . . . .                                                | 23 |
| 5.1.2 | Кодогенерация классов инструкций . . . . .                                                      | 23 |
| 5.2   | Регистровый файл и разрешение операндов . . . . .                                               | 24 |
| 5.2.1 | Структура регистрового файла . . . . .                                                          | 24 |
| 5.2.2 | Вспомогательные шаблоны <code>reg_cast&lt;T&gt;</code> и <code>to_u64&lt;T&gt;</code> . . . . . | 24 |
| 5.2.3 | Вычисление эффективного адреса . . . . .                                                        | 25 |
| 5.3   | Исполнители арифметических инструкций . . . . .                                                 | 25 |
| 5.3.1 | Целочисленная арифметика: формальная операционная семантика . . . . .                           | 25 |
| 5.3.2 | Арифметика с плавающей запятой: IEEE 754 и режимы округления . . . . .                          | 25 |
| 5.3.3 | Инструкции преобразования типов . . . . .                                                       | 26 |
| 5.4   | Исполнители инструкций работы с памятью . . . . .                                               | 26 |
| 5.4.1 | <code>ld.global / ld.shared</code> : чтение . . . . .                                           | 26 |
| 5.4.2 | <code>st.global / st.shared</code> : запись с учётом маски . . . . .                            | 27 |
| 5.4.3 | <code>setp / selp</code> : операции с предикатами . . . . .                                     | 27 |
| 5.5   | Исполнители инструкций управления потоком . . . . .                                             | 27 |
| 5.5.1 | Безусловный переход <code>bra.uni \$L</code> . . . . .                                          | 27 |
| 5.5.2 | Условный переход <code>%p bra \$L</code> . . . . .                                              | 27 |
| 5.5.3 | <code>bar.sync</code> : делегирование барьерной синхронизации . . . . .                         | 28 |
| 6     | Механизм управления дивергентным потоком . . . . .                                              | 29 |
| 6.1   | Теория PDOM-реконвергенции . . . . .                                                            | 29 |
| 6.1.1 | Граф управляющего потока и постдоминаторы . . . . .                                             | 29 |
| 6.1.2 | Теорема о корректности PDOM-реконвергенции . . . . .                                            | 29 |
| 6.2   | Структура данных стека дивергенции . . . . .                                                    | 30 |
| 6.2.1 | Тип <code>StackFrame</code> и семантика полей . . . . .                                         | 30 |
| 6.2.2 | Сложность операций и ограничение глубины . . . . .                                              | 30 |
| 6.3   | Алгоритм обработки условного перехода . . . . .                                                 | 31 |
| 6.3.1 | Алгоритм <code>PDOM_Diverge</code> . . . . .                                                    | 31 |
| 6.3.2 | Цикл проверки вершины стека . . . . .                                                           | 31 |
| 6.4   | Угловые случаи . . . . .                                                                        | 31 |
| 6.4.1 | Вложенные дивергенции: корректность при $k \geq 2$ . . . . .                                    | 31 |
| 6.4.2 | Дивергентные циклы . . . . .                                                                    | 32 |
| 6.4.3 | Ранний выход и переход к <code>isActive() = false</code> . . . . .                              | 32 |
| 7     | Система профилирования . . . . .                                                                | 33 |
| 7.1   | Архитектура профилировщика . . . . .                                                            | 33 |
| 7.1.1 | Кодогенерируемые профилировочные хуки . . . . .                                                 | 33 |
| 7.1.2 | Структура <code>ProfilingRecord</code> и буферизация . . . . .                                  | 34 |
| 7.1.3 | Метка времени: <code>steady_clock</code> . . . . .                                              | 34 |
| 7.2   | Формальные определения метрик . . . . .                                                         | 35 |
| 7.2.1 | <code>branch_efficiency</code> : эффективность выдачи . . . . .                                 | 35 |
| 7.2.2 | <code>bank_conflicts</code> : конфликты банков разделяемой памяти . . . . .                     | 35 |
| 7.2.3 | <code>global_coalescing</code> : коэффициент слияния транзакций . . . . .                       | 35 |
| 7.2.4 | <code>write_ops</code> : давление на регистровый файл . . . . .                                 | 36 |

|       |                                                                                                     |    |
|-------|-----------------------------------------------------------------------------------------------------|----|
| 7.3   | Инструмент генерации отчётов . . . . .                                                              | 36 |
| 7.3.1 | Парсер текстового вывода ( <code>parser.py</code> ) . . . . .                                       | 36 |
| 7.3.2 | PTX-парсер для отчёта ( <code>ptx_parser.py</code> ) . . . . .                                      | 36 |
| 7.3.3 | Агрегатор ( <code>aggregator.py</code> ): формулы накопления . . . . .                              | 37 |
| 7.3.4 | Рендерер HTML-отчёта ( <code>renderer.py</code> ) . . . . .                                         | 37 |
| 8     | Тестирование и верификация . . . . .                                                                | 38 |
| 8.1   | Модульные тесты . . . . .                                                                           | 38 |
| 8.2   | Часть I — Интеграционное тестирование CUDA-ядер . . . . .                                           | 38 |
| 8.2.1 | <code>vadd_custom</code> — поэлементное сложение векторов . . . . .                                 | 38 |
| 8.2.2 | <code>gelu_custom</code> — функция активации GELU . . . . .                                         | 39 |
| 8.2.3 | <code>softmax_custom</code> — онлайн-нормализация Softmax . . . . .                                 | 39 |
| 8.2.4 | <code>mmul_custom</code> — тайловое матричное умножение . . . . .                                   | 39 |
| 8.2.5 | <code>attention_custom</code> — Flash Attention-подобное ядро . . . . .                             | 40 |
| 8.2.6 | <code>fft_custom</code> — Radix-2 DIT FFT, $N = 32$ . . . . .                                       | 40 |
| 8.2.7 | <code>conjugate_gradient</code> — итерационный решатель СЛАУ . . . . .                              | 41 |
| 8.2.8 | Сводная таблица результатов интеграционных тестов . . . . .                                         | 41 |
| 8.3   | Часть II — Тестирование системы профилирования . . . . .                                            | 42 |
| 8.3.1 | Тест метрики <code>branch_efficiency</code> . . . . .                                               | 42 |
| 8.3.2 | Тест метрики <code>bank_conflicts</code> . . . . .                                                  | 42 |
| 8.3.3 | Тест метрики <code>global_coalescing</code> . . . . .                                               | 42 |
| 8.3.4 | Тест статистики регистров ( <code>write_ops</code> ) . . . . .                                      | 43 |
| 8.3.5 | Тест аннотации исходного кода через <code>.loc</code> . . . . .                                     | 43 |
| 8.3.6 | Тест агрегации: ручной расчёт vs. отчёт . . . . .                                                   | 43 |
| 9     | Заключение . . . . .                                                                                | 44 |
| 9.1   | Итоги работы . . . . .                                                                              | 44 |
| 9.2   | Ограничения . . . . .                                                                               | 44 |
| 9.3   | Направления развития . . . . .                                                                      | 45 |
| A     | Таблица поддерживаемых инструкций PTX . . . . .                                                     | 46 |
| B     | Формальная операционная семантика (полный список правил Structural Operational Semantics) . . . . . | 48 |
| C     | Трассировка PDOM_Diverge на примере вложенной дивергенции . . . . .                                 | 50 |
| D     | Формат файла <code>profiling.txt</code> : аннотированный пример . . . . .                           | 52 |
| E     | Структура HTML-отчёта: описание секций . . . . .                                                    | 54 |

# 1 Введение

## 1.1 Актуальность и мотивация исследования

Графические ускорители класса NVIDIA GPU заняли центральное место в современных высокопроизводительных вычислениях: от обучения крупных языковых моделей до физического моделирования и научных расчётов. Программирование GPU на платформе CUDA строится на модели SIMT (Single Instruction, Multiple Threads), в которой группы из 32 потоков — варпы — исполняют одну и ту же инструкцию над различными данными. Ключевым промежуточным представлением в этой экосистеме служит PTX (Parallel Thread Execution) — аппаратно-независимый ассемблер NVIDIA, в который компилируется CUDA C++ и из которого генерируется машинный код SASS для конкретного поколения GPU.

Несмотря на широкое распространение CUDA, отладка и профилирование CUDA-ядер по-прежнему сопряжены с существенными ограничениями. Официальные инструменты NVIDIA (Nsight, Compute Sanitizer) требуют наличия физического GPU, что делает их недоступными в типичных облачных CI-средах, виртуальных машинах и академических лабораториях с ограниченным бюджетом. Более того, аппаратные счётчики производительности предоставляют агрегированную статистику на уровне SM, но не дают детальных метрик на уровне отдельных инструкций PTX и строк исходного кода — что необходимо для глубокого анализа эффективности алгоритмов.

Программный интерпретатор PTX ISA позволяет устранить оба ограничения одновременно: работать на стандартном CPU без GPU-оборудования и собирать произвольные метрики уровня инструкций. Это открывает следующие сценарии применения:

1. CI-среды без GPU. Автоматическая проверка корректности CUDA-ядер при каждом коммите без специализированного оборудования.
2. Академические исследования. Изучение архитектурных свойств GPU-программ без доступа к аппаратным счётчикам.
3. Разработка и отладка алгоритмов. Детерминированное пошаговое исполнение с полным контролем над состоянием варпа.
4. Образовательные задачи. Визуальное представление механизмов дивергенции, слияния транзакций памяти и конфликтов банков для обучения принципам GPU-архитектуры.

## 1.2 Анализ существующих решений

На сегодняшний день существует ряд инструментов, частично решающих задачу функциональной эмуляции CUDA-ядер.

Ocelot [?] — один из первых функциональных PTX-интерпретаторов с поддержкой перехвата через CUDA Runtime API. Его ключевое ограничение — привязка к PTX ISA версий 1.x–2.x (CUDA 4.x) и отсутствие поддержки директив PTX 7.x+, в частности инструкций `ldmatrix`, `wmma` и атрибута `.address_size 64`. Проект не поддерживается с 2013 года.

GPGPU-Sim [?] — микроархитектурный симулятор, воспроизводящий конвейер SM с точностью до цикла (timing-accurate). Его цель принципиально отличается от задачи данной работы: GPGPU-Sim оценивает латентности и пропускную способность, тогда как функциональный интерпретатор нацелен на корректность результата и профилирование на уровне инструкций. Время одного запуска в GPGPU-Sim на порядки превышает время в функциональном эмуляторе.

Accel-Sim [?] — наследник GPGPU-Sim с поддержкой более новых архитектур, однако также является timing-accurate и требует конфигурационных файлов конкретного GPU.

CUDA Compute Sanitizer — официальный инструмент NVIDIA для обнаружения гонок данных, ошибок памяти и нарушений барьерной синхронизации. Требует наличия физического GPU и не предоставляет метрик уровня инструкций PTX.

Проведённый анализ выявляет исследовательскую нишу: не существует актуального (PTX ISA 7.x+), расширяемого функционального интерпретатора с интегрированным профилировщиком и механизмом прозрачного перехвата CUDA Runtime API без модификации бинарного кода приложения. Настоящая работа заполняет эту нишу.

### 1.3 Цели и задачи

Цель работы — разработка программного интерпретатора PTX ISA, обеспечивающего функционально корректное исполнение CUDA-ядер без GPU с возможностью сбора детальных метрик производительности на уровне инструкций.

Для достижения цели поставлены следующие задачи:

1. Реализовать механизм прозрачного перехвата CUDA Runtime API через LD\_PRELOAD с извлечением PTX посредством cuobjdump.
2. Разработать однократный инкрементный синтаксический анализатор PTX-текста, поддерживающий подмножество PTX ISA 7.x.
3. Построить таблицу диспетчеризации и исполнители инструкций с формальной операционной семантикой в нотации Structural Operational Semantics (SOS).
4. Реализовать алгоритм PDOM-реконвергенции для управления дивергентным потоком варпа.
5. Разработать кодогенерируемую систему метрик профилировщика с JSON-манифестом.
6. Реализовать инструмент генерации самодостаточного HTML-отчёта с аннотацией строк исходного кода.
7. Верифицировать корректность эмулятора на семи интеграционных тестах с реальными CUDA-ядрами.

### 1.4 Научная новизна

Научная новизна работы состоит в следующем:

1. Предложен и реализован прозрачный механизм перехвата CUDA Runtime API через LD\_PRELOAD, не требующий модификации бинарного файла приложения, с формальным обоснованием порядка разрешения символов динамическим загрузчиком.
2. Разработан однопроходный PTX-парсер с верифицированной корректностью на десяти CUDA-ядрах и 31 unit-тесте;
3. Сформулирована формальная операционная семантика подмножества PTX ISA, обеспечивающая верифицируемую корректность реализации исполнителей инструкций.
4. Предложена кодогенерируемая архитектура системы метрик, при которой добавление новой метрики сводится к правке JSON-манифеста без изменения C++-кода.
5. Дана формальная модель вычисления коэффициента слияния  $\kappa = T_{\text{ideal}}/T_{\text{actual}}$  для транзакций глобальной памяти с учётом размера элемента.

## 1.5 Практическая значимость

Разработанный инструмент имеет непосредственное практическое применение:

- Интеграция в CI-конвейеры. Эмулятор запускается как обычный Linux-процесс; для включения в `GitHub Actions` или `Jenkins` достаточно однострочного скрипта `суети ./binary`.
- Детальное профилирование. HTML-отчёт с аннотацией строк исходного кода позволяет определить «горячие» инструкции и оптимизировать паттерны доступа к памяти без запуска на GPU.
- Образовательная ценность. Пошаговая трассировка маски дивергенции позволяет наглядно продемонстрировать механизм SIMT-исполнения.

## 2 Теоретические основы

### 2.1 PTX как промежуточное представление CUDA

#### 2.1.1 Место PTX в стеке компиляции CUDA

Компиляция CUDA-программы проходит через многоуровневый конвейер. На входе компилятор `nvcc` принимает файл `.cu`, содержащий смешанный код: C++-код хоста и тела ядер устройства. Фронтенд выделяет код устройства и передаёт его в NVVM — LLVM-based IR для GPU — который транслируется в PTX.

PTX (Parallel Thread Execution) — аппаратно-независимое промежуточное представление, описывающее программу в терминах виртуального RISC-процессора с бесконечным регистровым файлом и явной параллельностью потоков. На следующем шаге `ptxas` транслирует PTX в SASS — бинарный машинный код конкретной архитектуры (Volta, Turing, Ampere, Hopper), который помещается в fatbinary-архив вместе с исходным PTX-текстом. При запуске CUDA Runtime JIT-компилирует PTX для установленного GPU при отсутствии подходящего SASS.

Fatbinary является ELF-совместимым архивом, встраиваемым компоновщиком в секцию `.nv_fatbin` результирующего ELF-файла приложения. Эмулятор извлекает PTX-текст из этой секции при перехвате `__cudaRegisterFatBinary`, вызывая утилиту `cuobjdump -dump-ptx` (подробно — в Главе 3).

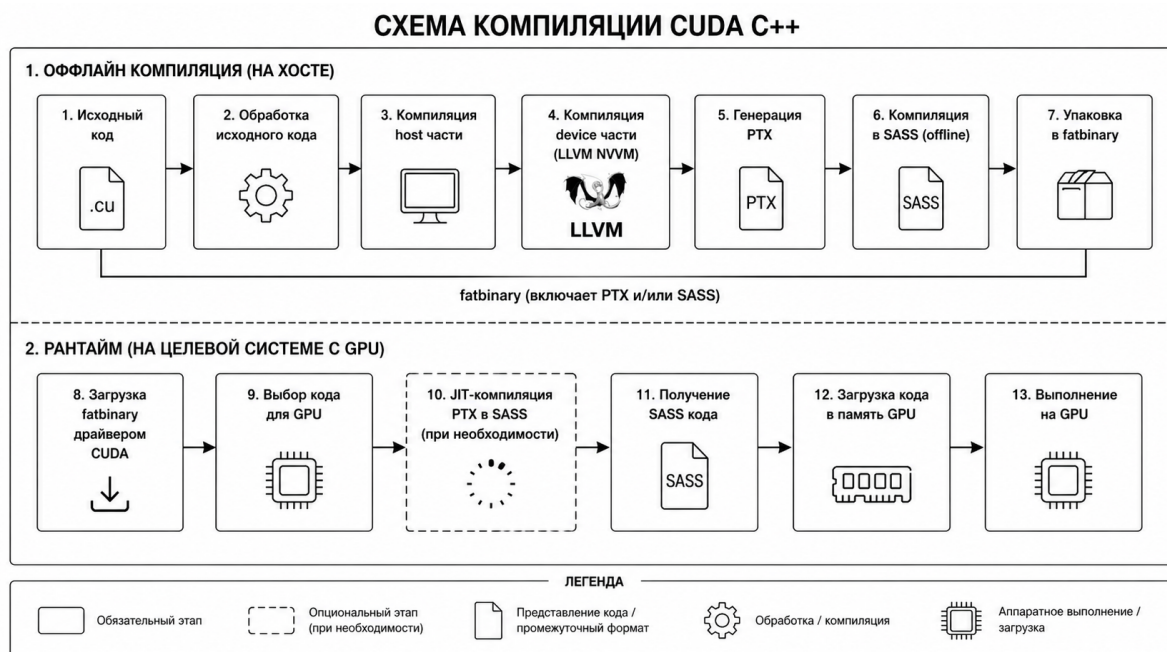


Рисунок 1: Стек компиляции CUDA. PTX занимает уровень между платформо-независимым IR и машинным кодом SASS.



### 2.1.2 Типовая система RTX

RTX определяет следующие базовые типы данных, каждый из которых однозначно задаётся суффиксом инструкции:

- Целочисленные знаковые: `s8`, `s16`, `s32`, `s64` — арифметика в дополнительном коде.
- Целочисленные беззнаковые: `u8`, `u16`, `u32`, `u64`.
- Битовые: `b8`, `b16`, `b32`, `b64`, `b128` — интерпретация не задана, используются для пересылок и упаковки.
- Вещественные: `f16`, `f32`, `f64` — форматы IEEE 754-2008 половинной, одинарной и двойной точности.
- Предикат: `pred` — однобитное логическое значение, хранимое в регистрах `%p0–%pN`.

Суффикс типа однозначно задаёт битовую ширину  $w \in \{8, 16, 32, 64, 128\}$  и способ интерпретации паттерна, что обеспечивает полную специфицируемость семантики каждой инструкции.

### 2.1.3 Классификация инструкций RTX

Для целей проектирования таблицы диспетчеризации все инструкции RTX разбиты на семь классов:

1. Арифметические (`add`, `sub`, `mul`, `div`, `fma`, `mad`, `rem`, `abs`, `neg`, `min`, `max`) — 25 мнемоник.
2. Логические и сдвиги (`and`, `or`, `xor`, `not`, `shl`, `shr`, `bfe`, `bfi`, `popc`) — 14 мнемоник.
3. Сравнения и предикаты (`setp`, `selp`, `slct`, `vote`) — 4 мнемоники.
4. Работа с памятью (`ld`, `st`, `mov`, `cvta`, `prefetch`) — 6 мнемоник.
5. Преобразование типов (`cvt`) — 1 мнемоника с матрицей суффиксов  $5 \times 5$ .
6. Управление потоком (`bra`, `call`, `ret`, `exit`, `trap`) — 5 мнемоник.
7. Синхронизация и специальные (`bar`, `membar`, `mov.special`, `ex2`, `lg2`, `rcp`, `sqrt`, `sin`, `cos`) — 11 мнемоник.

### 2.1.4 Директивы RTX и их роль в интерпретаторе

В RTX-тексте директивы не являются исполняемыми инструкциями, но несут информацию, критически необходимую для парсера:

- `.entry` / `.func` — задают тип функции (ядро устройства или вспомогательная функция) и её границы.

- `.param` — описывают сигнатуру: имя, тип и выравнивание каждого параметра ядра.
- `.reg .type %prefix<N>` — объявляют регистровый файл: например, `.reg .f32 %f<115>` резервирует 115 регистров типа `f32` с именами `%f0–%f114`.
- `.shared .align A .b8 name[S]` — резервирует  $S$  байт разделяемой памяти с выравниванием  $A$  байт.
- `.file ID "path"` и `.loc ID L C` — формируют таблицу соответствия RTX-инструкций строкам исходного кода, используемую профилировщиком для аннотации отчёта.

### 2.1.5 Пространства адресов и их семантика

RTX определяет шесть пространств адресов, отличающихся по видимости и времени жизни:

- `.reg` — регистровый файл, приватный для каждого потока.
- `.global` — глобальная память, разделяемая всеми блоками.
- `.shared` — разделяемая память, видимая всем потокам одного блока; время жизни — время жизни блока.
- `.local` — приватная память стека потока (спилл регистров).
- `.param` — область передачи параметров ядра с хоста.
- `.const` — константная память, только для чтения.

Формально вводится функция пространства:  $space : Addr \rightarrow \{global, shared, local, param, const\}$ , используемая исполнителями `ld` и `st` для выбора соответствующего обработчика обращения к памяти.

## 2.2 Математическая модель SIMT-исполнения

### 2.2.1 Варп как алгебраическая структура

Базовой единицей исполнения на GPU является варп — группа из 32 потоков, всегда исполняющих одну и ту же инструкцию. Формально варп определяется как кортеж:

$$W = \langle T, M, PC, R, S \rangle, \quad (1)$$

где:

- $T = \{0, 1, \dots, 31\}$  — множество идентификаторов потоков варпа;
- $M \in \mathbb{B}^{32}$  — маска исполнения (execution mask):  $M[i] = 1$  означает, что поток  $i$  исполняет целевую инструкцию;

- $PC \in \mathbb{N}$  — программный счётчик;
- $R$  — регистровый файл (функция  $T \times Types \times \mathbb{N} \rightarrow Val$ );
- $S$  — стек дивергенции (подробно в Главе 6).

Операция исполнения одной инструкции определяется как функция перехода состояния:  $\delta : W \times Instr \rightarrow W'$ .

### 2.2.2 Маска исполнения: операции и инварианты

Маска  $M \in \mathbb{B}^{32}$  формально задаёт, какие потоки исполняют следующую инструкцию. Определим вспомогательные операции:

$$\text{active}(M) = \{i \in T \mid M[i] = 1\}, \quad (2)$$

$$\text{popcount}(M) = |\text{active}(M)|, \quad (3)$$

$$\text{apply}(M, M') = M \wedge M' \quad (\text{маскирование по предикату}). \quad (4)$$

Фундаментальный инвариант исполнения: исполнитель инструкции обязан игнорировать потоки с  $M[i] = 0$  при записи результата. Нарушение этого инварианта является источником логических ошибок в реализации исполнителей и строго контролируется в тестах.

### 2.2.3 Предикатные регистры и составная маска

Предикатный регистр  $P \in \mathbb{B}^{32}$  хранит по одному биту на каждый поток варпа. Инструкция `@%p instr` выполняется с эффективной маской:

$$M_{\text{eff}} = M \wedge P, \quad (5)$$

а инструкция `@!%p instr` — с:

$$M_{\text{eff}} = M \wedge \neg P. \quad (6)$$

Инструкция `setp.op.type %p, a, b` вычисляет:

$$P[i] = \text{op}(a_i, b_i), \quad \forall i \in \text{active}(M), \quad (7)$$

что при следующем условном переходе `@%p bra` может привести к расщеплению варпа, если  $M \wedge P \neq 0$  и  $M \wedge \neg P \neq 0$ .

## 2.3 Математическая модель метрик производительности

### 2.3.1 Метрика `branch_efficiency`

Метрика эффективности ветвления определяется как доля активных потоков в момент выдачи инструкции:

$$\eta_{\text{branch}}(M) = \frac{\text{popcount}(M)}{|T|}, \quad (8)$$

где  $|T| = 32$  — размер варпа. Диапазон значений:  $\eta_{\text{branch}} \in [1/32, 1]$ , где  $\eta_{\text{branch}} = 1.0$  означает отсутствие дивергенции (все потоки активны), а  $\eta_{\text{branch}} = 1/32$  соответствует максимальной дивергенции (один активный поток).

Средняя эффективность по  $N$  выдачам:

$$\bar{\eta}_{\text{branch}} = \frac{1}{N} \sum_{k=1}^N \eta_{\text{branch}}(M_k). \quad (9)$$

### 2.3.2 Метрика `bank_conflicts`

Разделяемая память GPU организована в  $B$  банков шириной  $w_b$  байт каждый. Для обращения по адресу  $a$  функция принадлежности банку:

$$f(a) = \left\lfloor \frac{a}{w_b} \right\rfloor \bmod B. \quad (10)$$

Для множества активных потоков формируем мультимножество попаданий по банкам:  $\mathcal{G} = \{f(a_i) \mid i \in \text{active}(M)\}$ .

Число конфликтов:

$$C_{\text{bank}} = \max_{b \in \{0, \dots, B-1\}} |\{i \in \text{active}(M) : f(a_i) = b\}| - 1. \quad (11)$$

$C_{\text{bank}} = 0$  означает отсутствие конфликтов (каждый поток обращается к отдельному банку или к одному адресу — broadcast-режим).  $C_{\text{bank}} = 31$  соответствует конфликту 32 степени: все потоки обращаются к одному банку, что требует 32 последовательных транзакций.

### 2.3.3 Метрики `global_mem_transactions` и `global_coalescing`

Глобальная память обслуживается через кэш L1 с размером строки  $L = 128$  байт. Функция принадлежности кэш-линии:

$$\ell(a) = \left\lfloor \frac{a}{L} \right\rfloor. \quad (12)$$

Фактическое число транзакций равно числу уникальных кэш-линий, к которым обращаются активные потоки:

$$T_{\text{actual}} = |\{\ell(a_i) \mid i \in \text{active}(M)\}|. \quad (13)$$

Идеальное число транзакций при оптимальной упаковке:

$$T_{\text{ideal}} = \left\lceil \frac{\text{popcount}(M) \cdot s}{L} \right\rceil, \quad (14)$$

где  $s$  — размер элемента в байтах (например,  $s = 4$  для `f32`,  $s = 8$  для `f64`).

Коэффициент слияния:

$$\kappa = \frac{T_{\text{ideal}}}{T_{\text{actual}}} \in (0, 1]. \quad (15)$$

$\kappa = 1.0$  — идеальное слияние (все обращения покрываются минимальным числом транзакций);  $\kappa \approx 1/32$  — худший случай (каждый поток обращается к отдельной кэш-линии).

## 2.4 Операционная семантика РТХ (структурный подход)

### 2.4.1 Нотация малых шагов (small-step semantics)

Для формального описания семантики инструкций РТХ используется нотация структурной операционной семантики Structural Operational Semantics (SOS) [?]. Правило вида:

$$\langle instr, \sigma \rangle \rightarrow \sigma' \quad (16)$$

означает, что инструкция  $instr$  в состоянии  $\sigma = \langle M, R, PC, \dots \rangle$  выполняет переход в состояние  $\sigma'$  за один шаг.

Компоненты состояния:

- $R[t][\tau][k]$  — значение регистра номер  $k$  типа  $\tau$  потока  $t$ ;
- $M \in \mathbb{B}^{32}$  — текущая маска исполнения;
- $PC \in \mathbb{N}$  — программный счётчик.

### 2.4.2 Семантика арифметических инструкций

Для инструкции `add.s32 %rd, %ra, %rb`:

$$R'[t][s32][d] = (R[t][s32][a] + R[t][s32][b]) \bmod 2^{32}, \quad \forall t \in \text{active}(M). \quad (17)$$

Для инструкции `fma.rn.f32 %fd, %fa, %fb, %fc`:

$$R'[t][f32][d] = \text{round}_n(R[t][f32][a] \cdot R[t][f32][b] + R[t][f32][c]), \quad \forall t \in \text{active}(M), \quad (18)$$

где  $\text{round}_n$  — округление к ближайшему чётному (IEEE 754-2008, `roundTiesToEven`). Согласно стандарту, `fma` выполняет одно округление для всего выражения  $a \cdot b + c$  [?].

Правила определены только для потоков из  $\text{active}(M)$ ; состояние регистров неактивных потоков не изменяется.

### 2.4.3 Семантика инструкций сравнения и выбора

Для `setp.lt.s32 %p, %ra, %rb`:

$$P'[t] = (R[t][s32][a] < R[t][s32][b]), \quad \forall t \in \text{active}(M). \quad (19)$$

Для `sefp.f32 %fd, %fa, %fb, %p`:

$$R'[t][f32][d] = \begin{cases} R[t][f32][a] & \text{если } P[t] = 1, \\ R[t][f32][b] & \text{если } P[t] = 0, \end{cases} \quad \forall t \in \text{active}(M). \quad (20)$$

Следует отметить, что `sefp` не вызывает дивергенцию: оба операнда вычисляются внутри каждого потока без расщепления маски.

#### 2.4.4 Семантика инструкций управления потоком

Для безусловного перехода `bra.uni $L`:

$$\langle \text{bra.uni } \$L, \sigma \rangle \rightarrow \sigma[\text{PC} \leftarrow \text{offset}(\$L)], \quad (21)$$

где  $\text{offset}(\$L)$  — PC первой инструкции метки  $\$L$  из таблицы базовых блоков; маска  $M$  не изменяется.

Для условного перехода `@%p bra $L` определим:  $M_{\text{taken}} = M \wedge P$ ,  $M_{\text{not}} = M \wedge \neg P$ .

- Случай 1 ( $M_{\text{not}} = 0$ ): все активные потоки берут переход.  $\text{PC}' = \text{offset}(\$L)$ ,  $M' = M$ .
- Случай 2 ( $M_{\text{taken}} = 0$ ): ни один активный поток не берёт переход.  $\text{PC}' = \text{PC} + 1$ ,  $M' = M$ .
- Случай 3 ( $M_{\text{taken}} \neq 0 \wedge M_{\text{not}} \neq 0$ ): дивергенция. Применяется алгоритм `PDOM_Diverge` (подробно в Главе 6).

#### 2.4.5 Семантика инструкций памяти

Для `ld.global.f32 %fd, [%ra]`:

$$R'[t][f32][d] = \text{Mem}_{\text{global}}[R[t][u64][a]], \quad \forall t \in \text{active}(M), \quad (22)$$

где  $\text{Mem}_{\text{global}}$  — функция чтения глобальной памяти, реализованная компонентами контекста исполнения (Часть 2).

После исполнения `ld.global` / `st.global` вычисляются  $T_{\text{actual}}$  и  $\kappa$  по формулам (39)–(41).

## 2.5 Механизм динамической подстановки библиотек в Linux

### 2.5.1 Загрузчик ELF и таблица динамических символов

При запуске ELF-бинарного файла в Linux ядро передаёт управление динамическому загрузчику `ld.so`. Загрузчик строит список разрешения символов — упорядоченную цепочку библиотек, в которой он ищет адрес каждого используемого символа [? ].

При помещении `libemuruntime.so` на позицию  $l_1$  (через `LD_PRELOAD=libemuruntime.so`) все вызовы `cudaMalloc`, `cudaLaunchKernel` и прочих функций CUDA Runtime разрешаются в реализации эмулятора по формуле (?), не достигая оригинальной `libcudart.so`. При этом сам бинарный файл приложения остаётся немодифицированным.

## 2.6 Обзор смежных работ

### 2.6.1 Функциональные эмуляторы: Ocelot, CUDAsim

Проект Ocelot [?] разработан в Технологическом институте Джорджии в 2009–2013 гг. Он реализует функциональную эмуляцию PTX с поддержкой перехвата CUDA 4.x API через механизм, аналогичный `LD_PRELOAD`. Ocelot не поддерживает PTX ISA 7.x+ (инструкции `ldmatrix`, `wmma`, `async.bulk.copy`) и не компилируется с современными версиями GCC/Clang. Профилировочные возможности ограничены счётчиком выполненных инструкций.

CUDAsim (CUDA Device Emulation, устаревший режим) — встроенный в CUDA 4.x режим эмуляции CPU, удалённый в CUDA 5.0 из-за расхождений в поведении атомарных операций и барьеров.

### 2.6.2 Микроархитектурные симуляторы: GPGPU-Sim, Accel-Sim

GPGPU-Sim [?] — timing-accurate симулятор, воспроизводящий пайплайн SM с точностью до цикла: frontend, выдачу инструкций, исполнительные блоки, кэши, DRAM. Его цель принципиально отличается: оценка латентностей и пропускной способности, а не функциональная корректность в CI-контексте. Один запуск `mmul_custom` ( $64 \times 64 \times 64$ ) в GPGPU-Sim занимает порядка 10 часов машинного времени.

Accel-Sim [?] — расширение GPGPU-Sim с поддержкой архитектур Volta/Turing/Ampere, трассировкой SASS через NVBit и дополнительными конфигурационными файлами.

Сравнение подходов приведено в таблице 1.

Таблица 1: Сравнение инструментов эмуляции/симуляции CUDA

| Инструмент        | PTX ISA | Перехват     | Профилировщик | Тип            |
|-------------------|---------|--------------|---------------|----------------|
| Ocelot            | 1.x–2.x | LD_PRELOAD   | Счётчики      | Функциональный |
| GPGPU-Sim         | < 7.x   | Не требуется | Timing        | Архитектурный  |
| Accel-Sim         | < 8.x   | NVBit trace  | Timing        | Архитектурный  |
| Compute Sanitizer | 8.x     | Требует GPU  | Нет           | Аппаратный     |
| Данная работа     | 7.x     | LD_PRELOAD   | 4 метрики     | Функциональный |

## 3 Механизм перехвата CUDA Runtime

### 3.1 Архитектура перехвата через LD\_PRELOAD

#### 3.1.1 Схема взаимодействия компонентов

Центральная идея механизма перехвата состоит в том, что CUDA-приложение вызывает функции из `libcudart.so` через PLT, а динамический загрузчик `ld.so` разрешает эти символы в реализации из `libemuruntime.so`, заблаговременно помещённой в переменную `LD_PRELOAD`.

Полная схема взаимодействия компонентов:

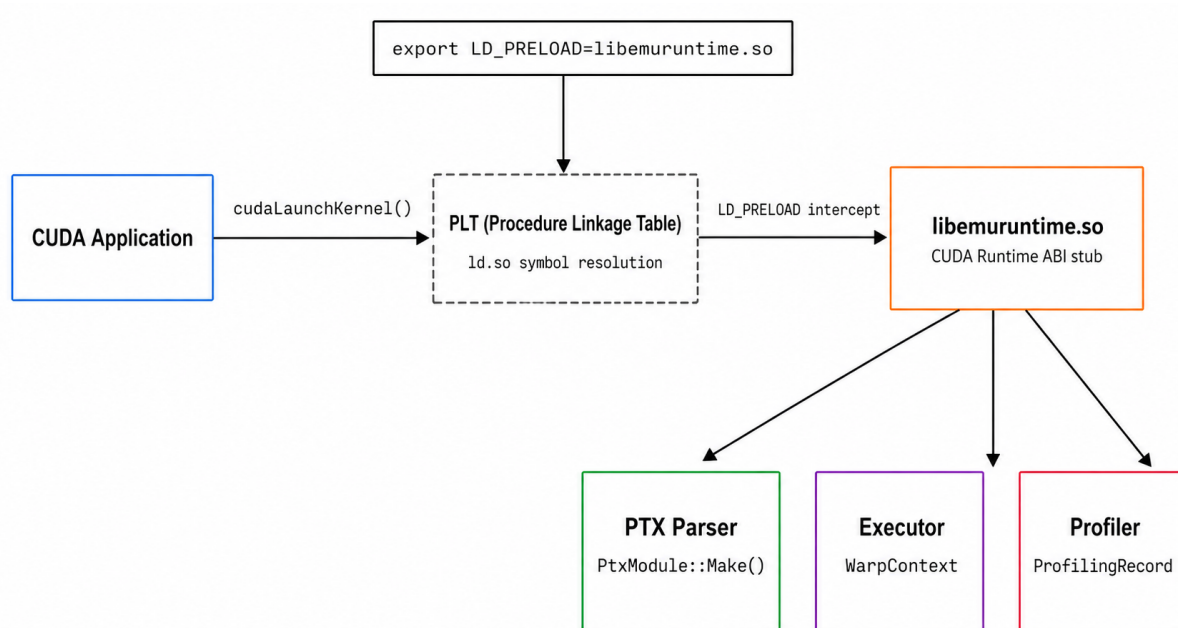


Рисунок 2: Схема перехвата CUDA Runtime через LD\_PRELOAD.

Библиотека `libemuruntime.so` реализует полный публичный ABI `libcudart.so`, необходимый для работы типичного CUDA-приложения. Каждая функция-заглушка либо делегирует работу компонентам эмулятора (парсеру, исполнителю, профилировщику), либо возвращает `cudaSuccess` для функций, не требующих реальной реализации в пределах эмулятора (например, управление событиями, профили потоков).

#### 3.1.2 Реализованные точки перехвата CUDA Runtime API

В таблице 2 перечислены перехватываемые функции CUDA Runtime API с указанием ответственного компонента.



Таблица 2: Реализованные точки перехвата CUDA Runtime API

| Функция                              | Назначение                                                          | Компонент |
|--------------------------------------|---------------------------------------------------------------------|-----------|
| <code>__cudaRegisterFatBinary</code> | Регистрация модуля; вызов <code>cuobjdump</code> для извлечения PTX | Часть 1   |
| <code>__cudaRegisterFunction</code>  | Отображение дескриптор $\rightarrow$ имя ядра                       | Часть 1   |
| <code>cudaLaunchKernel</code>        | Запуск ядра через диспетчер                                         | Часть 2   |
| <code>cudaMalloc</code>              | Выделение памяти устройства                                         | Часть 2   |
| <code>cudaFree</code>                | Освобождение памяти устройства                                      | Часть 2   |
| <code>cudaMemcpy</code>              | Копирование памяти хост $\leftrightarrow$ устройство                | Часть 2   |
| <code>cudaMemset</code>              | Инициализация памяти                                                | Часть 2   |
| <code>cudaDeviceSynchronize</code>   | Синхронизация (по-оп в эмуляторе)                                   | Часть 2   |

### 3.1.3 Извлечение PTX-текста через `cuobjdump`

При перехвате `__cudaRegisterFatBinary(handle)` эмулятор не разбирает `fatbinary` напрямую. Вместо этого он определяет путь к исполняемому файлу приложения из переменной окружения `CUEMU_TARGET_EXEC` и передаёт его утилите `cuobjdump`:

Листинг 1: Извлечение PTX через `cuobjdump`

```
cuobjdump --dump-ptx <binary> > ptx_text.txt
```

`cuobjdump` извлекает PTX-текст из секции `.nv_fatbin` ELF-файла и выводит его в стандартный вывод. Эмулятор читает этот вывод и передаёт PTX-текст в `PtxModule::Make()`, которая строит таблицу инструкций. Полученный `Module` сохраняется в `GlobalContext` под ключом-дескриптором.

Алгоритм извлечения PTX:

1. Прочитать путь к бинарному файлу из `CUEMU_TARGET_EXEC`.
2. Выполнить `cuobjdump -dump-ptx <binary>`, захватить `stdout` через `popen()`.
3. Передать полученный текст в `PtxModule::Make()`, получить объект `Module` с таблицей инструкций.
4. Сохранить `Module` в `GlobalContext` под ключом-дескриптором.

### 3.1.4 Регистрация функций: отображение дескриптор $\rightarrow$ имя ядра

CUDA Runtime ABI определяет, что при вызове `cudaLaunchKernel` ядро идентифицируется указателем на функцию-заглушку в сегменте кода, а не строковым именем. Это обусловлено тем, что в C++ ядра компилируются в обычные функции, и CUDA не гарантирует сохранение символьных имён после компоновки.

При перехвате `__cudaRegisterFunction` эмулятор создаёт отображение:

$$functions\_ [ptr] = name, \quad (23)$$

где `ptr` — числовое представление указателя на заглушку, `name` — строка вида `"_Z11vadd_kernelPfS_S_i"`.

При последующем вызове `cudaLaunchKernel(ptr, ...)` эмулятор извлекает строковое имя из отображения по ключу `ptr` и напрямую ищет функцию в `PtxModule::GetEntryFunction(name)` без демангла — PTX-имя уже является декодированным символом, которым `cuobjdump` подписывает каждую точку входа.

### 3.1.5 Запуск `cuemu`: CLI-лончер и переменные окружения

CLI-инструмент `cuemu` является тонким лончером: он подготавливает окружение и повторно запускает целевой бинарный файл:

Листинг 2: Логика запуска `cuemu`

```
export LD_PRELOAD=$CUEMU_LIB_DIR/libemuruntime.so
export LD_LIBRARY_PATH=$CUEMU_LIB_DIR:$LD_LIBRARY_PATH
export CUEMU_TARGET_EXEC="$1"
export CUEMU_COLLECT_PROFILING="${CUEMU_COLLECT_PROFILING:-0}"
export CUEMU_PROFILING_OUTPUT="${CUEMU_PROFILING_OUTPUT:-profiling.txt}"
exec "$@"
```

Поддерживаемые переменные окружения перечислены в таблице 3.

Таблица 3: Переменные окружения эмулятора

| Переменная                           | Назначение                   | По умолчанию               |
|--------------------------------------|------------------------------|----------------------------|
| <code>CUEMU_COLLECT_PROFILING</code> | Включить сбор метрик         | 0                          |
| <code>CUEMU_PROFILING_OUTPUT</code>  | Путь к файлу вывода          | <code>profiling.txt</code> |
| <code>CUEMU_GPU_CONFIG</code>        | Путь к TOML-конфигу GPU      | встроенный                 |
| <code>CUEMU_TARGET_EXEC</code>       | Имя целевого бинарного файла | обязателен                 |

## 4 Синтаксический анализатор РТХ

### 4.1 Требования к парсеру и выбор стратегии

#### 4.1.1 Анализ структуры РТХ-текста

РТХ-текст имеет строго линейную структуру: последовательность директив, за которыми следуют тела функций. Каждое тело функции — последовательность базовых блоков; каждый базовый блок — последовательность инструкций, предваряемых необязательной меткой. В отличие от языков программирования общего назначения, РТХ не содержит вложенных выражений или скобочных структур внутри отдельных инструкций.

Эти свойства обосновывают отказ от построения полного AST и использование однопроходного линейного разбора. Сложность:  $O(N)$  по числу строк  $N$ , что критически важно для РТХ-файлов размером 5–50 КБ, возникающих при встраивании трансцендентных функций компилятором NVCC.

#### 4.1.2 Однопроходный инкрементный разбор

Парсер реализует конечный автомат с тремя состояниями:  $Q = \{q_{\text{init}}, q_{\text{func}}, q_{\text{block}}\}$  и следующими переходами:

- $q_{\text{init}} \xrightarrow{\text{.entry/.func}} q_{\text{func}}$ : встретили заголовок функции.
- $q_{\text{func}} \xrightarrow{\text{\$label:}} q_{\text{block}}$ : встретили метку базового блока.
- $q_{\text{block}} \xrightarrow{\text{instruction}} q_{\text{block}}$ : добавляем инструкцию в текущий блок.
- $q_{\text{block}} \xrightarrow{\text{}} q_{\text{init}}$ : закрывающая скобка тела функции.

На каждом шаге поддерживается контекст: текущая функция, текущий базовый блок, таблица файлов `source_file/source_line`, монотонный счётчик РС.

Таким образом, однопроходный инкрементный разбор обеспечивает линейную сложность  $O(N)$  по числу строк РТХ-текста и полное отсутствие возвратов. Результатом является структурированный объект `PtxModule`, содержащий таблицу инструкций с монотонными РС-адресами, граф базовых блоков и таблицу символов для разрешения переходов во время исполнения.

### 4.2 Реализация парсера

#### 4.2.1 Разбор объявлений функций и параметров

Заголовок функции разбирается регулярным выражением:

### Листинг 3: Регулярное выражение для заголовка функции

```
static const std::regex kFuncRe{
    R"(\.(?:entry|func)\s+)"           // kind
    R"((?:\(.*\?\)\s*)?)"             // optional return type
    R"((\w+))"                         // name
    R"(\s*\(((\[^\]]*\))\))",          // params
    std::regex::optimize
};
```

Результат — объект `Function` с таблицей параметров `params_` и флагом `isEntry()`, по которому диспетчер различает точки входа ядра и вспомогательные функции. Параметры функции разбираются рекурсивно: каждый элемент вида `.param .align N .bM name` порождает запись `ParamSlot{offset, size}`, используемую исполнителем при копировании аргументов вызова в регистровый файл.

Директива `.reg .type %prefix<N>` объявляет  $N$  регистров заданного типа. Парсер сохраняет: `reg_decls[func][prefix] = (count, type)`. Эти данные используются исполнителем при инициализации регистрового файла варпа — выделяется ровно столько слотов, сколько объявлено в PTX, без избыточного выделения. Профилировщик дополнительно использует эту таблицу для подсчёта `write_ops` по префиксу.

#### 4.2.2 Таблица меток и граф базовых блоков

Таблица `BasicBlockAddrMap` является ключевой структурой, связывающей парсер с исполнителем управляющего потока. Метка вида `$BB0_3`: фиксирует точку входа в базовый блок:

1. Закрываем текущий базовый блок, сохраняем его в `PtxModule`.
2. Записываем `basic_blocks["$BB0_3"] = current_pc` — отображение имени метки в монотонный РС первой инструкции блока.
3. Открываем новый базовый блок.

При исполнении инструкции ветвления `bra $L` исполнитель вызывает `GetBasicBlockOffset("$L")` и устанавливает РС варпа в полученное значение. Без этой таблицы реализация переходов потребовала бы линейного поиска по всей таблице инструкций.

#### 4.2.3 Разбор отдельной инструкции

Каждая строка-инструкция разбирается в пять последовательных этапов, полностью определяющих данные, необходимые исполнителю во время выполнения:

1. Предикатный префикс. Опциональный `@%pN` или `@!%pN` извлекается первым и сохраняется в поле `predicate`. Исполнитель проверяет его перед выполнением: если предикатный регистр ложен, инструкция пропускается без изменения состояния регистрового файла.

2. Нормализация мнемоники. `fma.rn.f32` разбивается на ключ поиска в таблице диспетчеризации (`fma`) и список модификаторов `["rn "f32"]`, которые передаются исполнителю для выбора режима округления и ширины типа.
3. Список операндов. Строка разбивается по запятой с учётом скобок `[...]`, порождающая упорядоченный вектор `Operand`. Порядок критичен: первый операнд — регистр назначения для большинства инструкций.
4. Классификация и разрешение операндов (подробно — в следующем разделе).
5. Присвоение монотонного РС. Счётчик инкрементируется для каждой исполняемой инструкции и используется как ключ в `instruction_table_` и в записях `profiling.txt`.

#### 4.2.4 Разрешение операндов

Классификация каждого операнда производится единожды при разборе, а не при каждом исполнении инструкции, что устраняет накладные расходы на разбор строк в горячем цикле исполнителя:

- Именованный регистр: `%r5`, `%f12`, `%rd0`, `%p3`. Префикс однозначно определяет тип (`r` → `s32`, `f` → `f32`, `rd` → `u64`, `p` → `pred`) и индекс слота в регистровом файле варпа.
- Числовой литерал: десятичный (`42`), шестнадцатеричный (`0x3f800000`) или IEEE 754 (`0f3f800000`). Значение конвертируется в `uint64_t` при разборе и хранится непосредственно в `Operand`.
- Адресное выражение: `[%rd5+8]` — базовый регистр и смещение разделяются при разборе; исполнитель вычисляет адрес как `reg[base] + imm` без дополнительного разбора строки.
- Символическая метка: `$BB0_3` — хранится как строка; при первом исполнении перехода разрешается через `BasicBlockAddrMap` и кэшируется.

### 4.3 Внутреннее представление инструкций

#### 4.3.1 Иерархия классов инструкций и монотонный счётчик РС

Каждая RTX-инструкция представлена отдельным классом, наследующим абстрактный базовый класс:

Листинг 4: Базовый класс (`build/app/ptx_asm/autogen/instructions.h`)

```
class Instruction {
public:
    virtual void Execute(std::shared_ptr<WarpContext>& wc) = 0;
    virtual std::string_view Name() const = 0;
    virtual bool HasExplicitPredicate() const { return false; }
};
```

Конкретные подклассы генерируются из `instructions.json`. Каждый хранит только поля, специфичные для своей мнемоники (пространство памяти, тип данных, режим округления, операнды):

Листинг 5: Пример: класс `ldInstruction`

```
class ldInstruction : public Instruction {
public:
    ldspaceQl  space_; // Global, Shared, Local, Const
    dataType  data_;  // F32, F64, U32, ...
    addressOperand addr_; // base register + immediate offset
    registerOperand dst_; // destination register
    std::optional<registerOperand> pred_; // predicate

    void Execute(std::shared_ptr<WarpContext>& wc) override;
    std::string_view Name() const override { return "ld"; }
    bool HasExplicitPredicate() const override;
};
```

Монотонный PC присваивается инструкции при разборе PTX и хранится в `InstructionList` модуля как индекс в глобальном векторе: `Module::GetInstruction(pc)` возвращает `shared_ptr<Instruction>` за  $O(1)$ . Тот же PC используется как ключ в записях `profiling.txt`, обеспечивая инъективное соответствие между трассой и таблицей инструкций.

При исполнении `Module::GetInstruction(pc)` возвращает `const Instruction&` за  $O(1)$  через `instruction_table_[pc]`.

## 5 Система исполнения инструкций

### 5.1 Таблица диспетчеризации

#### 5.1.1 Структура: фабрика и виртуальный вызов

Каждая PTX-инструкция представлена отдельным C++-классом, наследующим абстрактный базовый класс `Instruction`:

Листинг 6: Базовый класс инструкции

```
class Instruction {
public:
    virtual void Execute(std::shared_ptr<WarpContext>& wc) = 0;
    virtual std::string_view Name() const = 0;
    virtual bool HasExplicitPredicate() const { return false; }
};
```

Создание объекта нужного подкласса выполняется фабричной функцией `makeInstruction(name, line)`, которую генерирует `autogen.py` из `instructions.json`:

Листинг 7: Фрагмент автогенерируемой фабрики

```
std::shared_ptr<Instruction> makeInstruction(
    const std::string& name, const std::string& line) {
    if (name == "add") return addInstruction::Make(line);
    if (name == "fma") return fmaInstruction::Make(line);
    if (name == "ld") return ldInstruction::Make(line);
    // ... all supported instructions ...
}
```

Фабрика вызывается один раз при разборе PTX-текста. В горячем цикле исполнения варпа производится единственный виртуальный вызов: `instr->Execute(warp_ctx)`. Накладные расходы — одна косвенная адресация через `vtable`.

#### 5.1.2 Кодогенерация классов инструкций

Скрипт `autogen.py` читает `instructions.json` и генерирует файлы `instructions.h/.cpp` с классом для каждой мнемоники. Единственный источник истины в JSON предотвращает рассинхронизацию между парсером, фабрикой и тестами.

Принцип открытости/закрытости (ОСР) [?]: добавление поддержки новой инструкции сводится к добавлению записи в `instructions.json` и реализации метода `Execute()` — `autogen.py` вставит нужную ветку в фабрику автоматически.

## 5.2 Регистровый файл и разрешение операндов

### 5.2.1 Структура регистрового файла

Регистровый файл варпа организован как вектор на поток, где для каждого потока хранится отображение типа регистра на вектор значений:

Листинг 8: Структура регистрового файла

```
using RegisterContext = std::vector<uint64_t>;
using ThreadContext = std::unordered_map<Ptx::registerType,
                                         RegisterContext>;
// thread_regs[thread_id][registerType][slot]
std::vector<ThreadContext> thread_regs;
```

Ключ `registerType` — перечисление {R, Rd, F, Fd, P, ...}; вектор значений индексирован номером регистра из PTX-операнда. Все значения хранятся в `uint64_t`: покрывает `f64` без потерь; значения `f32` занимают младшие 32 бита. Преобразование выполняется через `std::bit_cast` без нарушения правила `strict aliasing` (C++20).

Размер внутренних векторов определяется директивами `.reg` в PTX: исполнитель выделяет ровно столько слотов, сколько объявлено, без избыточного выделения памяти.

### 5.2.2 Вспомогательные шаблоны `reg_cast<T>` и `to_u64<T>`

Все исполнители инструкций работают с регистровым файлом через два шаблонных примитива, определённых в `executors.hpp`:

Листинг 9: Примитивы доступа к регистрам

```
template<typename T>
T reg_cast(uint64_t raw) {
    if constexpr (std::is_integral_v<T>)
        return static_cast<T>(raw);
    else {
        T val;
        std::memcpy(&val, &raw, sizeof(T));
        return val;
    }
}

template<typename T>
uint64_t to_u64(T val) {
    if constexpr (std::is_integral_v<T>)
        return static_cast<uint64_t>(
            static_cast<std::make_unsigned_t<T>>(val));
    else {
        uint64_t raw = 0;
        std::memcpy(&raw, &val, sizeof(T));
        return raw;
    }
}
```



Целые числа преобразуются через `static_cast` через беззнаковый промежуточный тип — бит-паттерн знаковых значений в `uint64_t` сохраняется. Числа с плавающей запятой сериализуются через `memcpu` — это единственный способ избежать UB по правилу `strict aliasing` в C++.

### 5.2.3 Вычисление эффективного адреса

Эффективный адрес потока  $t$  для операнда `[%rd5+8]`:

$$ea(t) = R[t][u64][5] + 8, \quad (24)$$

где `imm_offset = 8` извлекается парсером из адресного выражения. Вычисленный адрес  $ea(t)$  передаётся в делегирующий метод `WarpContext`, который маршрутизирует обращение в нужное пространство памяти на основе функции  $space(ea(t))$ .

## 5.3 Исполнители арифметических инструкций

### 5.3.1 Целочисленная арифметика: формальная операционная семантика

Правила Structural Operational Semantics (SOS) для ключевых целочисленных инструкций:

`add.u32 %rd, %ra, %rb:`

$$R'[t][u32][d] = (R[t][u32][a] + R[t][u32][b]) \bmod 2^{32}, \quad \forall t \in \text{active}(M). \quad (25)$$

`mul.hi.u32 %rd, %ra, %rb:`

$$R'[t][u32][d] = \left\lfloor \frac{R[t][u32][a] \cdot R[t][u32][b]}{2^{32}} \right\rfloor, \quad \forall t \in \text{active}(M). \quad (26)$$

`div.s32 %rd, %ra, %rb:`

$$R'[t][s32][d] = \text{trunc} \left( \frac{R[t][s32][a]}{R[t][s32][b]} \right), \quad \forall t \in \text{active}(M), R[t][s32][b] \neq 0. \quad (27)$$

Граничные случаи: деление на ноль — по спецификации РТХ возвращает 0; переполнение `INT_MIN / -1` — возвращает `INT_MIN` (UB в C++, явно обрабатывается).

### 5.3.2 Арифметика с плавающей запятой: IEEE 754 и режимы округления

РТХ определяет четыре модификатора округления, соответствующие режимам IEEE 754-2008 [? ]:

Модификаторы округления (`.rn`, `.rz`, `.ru`, `.rd`) разбираются парсером и сохраняются в объекте инструкции. В текущей реализации эмулятор не переключает режим округления хост-CPU через `fesetround`: операции с плавающей запятой выполняются с режимом `FE_TONEAREST` по умолчанию. Это известное ограничение, которое не влияет на корректность результата для инструкций, не чувствительных к последнему биту (`faithful rounding`).

Таблица 4: Режимы округления PTX и их соответствие IEEE 754

| Суффикс PTX | Режим IEEE 754      | Описание             |
|-------------|---------------------|----------------------|
| .rn         | roundTiesToEven     | к ближайшему чётному |
| .rz         | roundTowardZero     | усечение             |
| .ru         | roundTowardPositive | к $+\infty$          |
| .rd         | roundTowardNegative | к $-\infty$          |

### 5.3.3 Инструкции преобразования типов

`cvt.rni.s32.f32 %rd, %fa`: преобразование  $f32 \rightarrow s32$  с округлением к ближайшему целому:

$$R'[t][s32][d] = \lfloor R[t][f32][a] + 0.5 \rfloor, \quad \forall t \in \text{active}(M). \quad (28)$$

`cvt.rz.f32.s64 %fd, %ra`: преобразование  $s64 \rightarrow f32$  с усечением дробной части:

$$R'[t][f32][d] = \text{round}_z(R[t][s64][a]), \quad \forall t \in \text{active}(M). \quad (29)$$

## 5.4 Исполнители инструкций работы с памятью

### 5.4.1 `ld.global` / `ld.shared`: чтение

Исполнитель `ld` реализован как шаблонный метод `ldInstruction::ExecuteThread<dataType Data>`, параметризованный типом данных. Тип `ptx_native_t<Data>` — псевдоним, отображающий перечислитель `dataType` на соответствующий C++-тип ( $F32 \rightarrow \text{float}$ ,  $S64 \rightarrow \text{int64\_t}$  и т.д.):

Листинг 10: `ldInstruction::ExecuteThread` (`executors.tpp`)

```
template<dataType Data>
void ldInstruction::ExecuteThread(
    uint32_t lid, std::shared_ptr<WarpContext>& wc)
{
    using T = ptx_native_t<Data>;
    uintptr_t addr = 0;
    if (addr_.reg.type != registerType::UNDEFINED) {
        addr = static_cast<uintptr_t>(
            wc->thread_regs[lid][addr_.reg.type][addr_.reg.reg_id]);
        addr += static_cast<ptrdiff_t>(addr_.imm);
    }
    if (space_ == ldspaceQ1::Shared) {
        auto* base = static_cast<uint8_t*>(wc->getSharedBase());
        addr = reinterpret_cast<uintptr_t>(base + addr);
    }
    T val = T{};
    std::memcpy(&val, reinterpret_cast<const void*>(addr), sizeof(T));
    wc->thread_regs[lid][dst_.type][dst_.reg_id] = to_u64<T>(val);
}
```

Метод вызывается по одному разу для каждого активного потока варпа. Переключение между пространствами памяти (`global/shared`) производится внутри метода по полю `space_`, разобранному парсером. Данные копируются через `memcpy` — без нарушения `strict aliasing` и без промежуточного приведения указателей.

#### 5.4.2 `st.global` / `st.shared`: запись с учётом маски

Для каждого потока  $t \in \text{active}(M)$ :  $*ptr(ea(t)) \leftarrow R[t][\tau][src]$ . Потoki с  $M[i] = 0$  пропускают запись — это семантически обязательно: неактивные потоки не должны изменять состояние памяти даже при вычислении адреса.

#### 5.4.3 `setp` / `selp`: операции с предикатами

Для `setp.lt.f32 %p, %fa, %fb`:

$$P'[t] = (R[t][f32][a] < R[t][f32][b]), \quad \forall t \in \text{active}(M). \quad (30)$$

Результат сравнения (`true/false`) записывается в один предикатный регистр `dst_` как значение 1/0 типа `uint64_t`.

### 5.5 Исполнители инструкций управления потоком

#### 5.5.1 Безусловный переход `bra.uni $L`

$$\langle \text{bra.uni } \$L, \sigma \rangle \rightarrow \sigma[\text{PC} \leftarrow \text{offset}(\$L)]. \quad (31)$$

Суффикс `.uni` (`uniform`) гарантирует конвергентность. В реализации: `warp.pc = warp.GetBasicBlockPc("$L")`. Стек дивергенции не модифицируется.

#### 5.5.2 Условный переход `@%p bra $L`

Полная семантика с разбором по случаям (подробно в Главе 6):

$$M_{\text{taken}} = M \wedge P, \quad M_{\text{not}} = M \wedge \neg P.$$

- Случай 1 ( $M_{\text{not}} = 0$ ):  $\text{PC}' = \text{offset}(\$L)$ .
- Случай 2 ( $M_{\text{taken}} = 0$ ):  $\text{PC}' = \text{PC} + 1$ .
- Случай 3: Вызов `PDOM_Diverge` (см. Главу 6).

Инструкция `ret` завершает исполнение варпа: она сбрасывает устаревшие фреймы `PDOM`-стека дивергенции и устанавливает `pc = EOC` (End-of-Code), сигнализируя циклу исполнения об окончании:

Листинг 11: `retInstruction::ExecuteBranch` (`executors.tpp`)

```
// Discard stale convergence frames from guard-clause paths.
while (!wc->execution_stack.empty()
      && wc->execution_stack.top().is_convergence)
    wc->execution_stack.pop();

if (wc->execution_stack.empty())
    wc->pc = WarpContext::EOC;
else {
    const auto frame = wc->execution_stack.top();
    wc->execution_stack.pop();
    wc->pc = frame.pc;
    wc->execution_mask = frame.mask;
}
```

### 5.5.3 `bar.sync`: делегирование барьерной синхронизации

Исполнитель `bar.sync` `N` вызывает: / `warp.syncBarrier()`, реализованный в компонентах управления блоком (Часть 2).

Математически: все записи `st.shared` до `bar.sync` гарантированно видимы всем потокам после него — барьер обеспечивает полную видимость памяти (`happens-before` по отношению к записям разделяемой памяти).

## 6 Механизм управления дивергентным потоком

### 6.1 Теория PDOM-реконвергенции

#### 6.1.1 Граф управляющего потока и постдоминаторы

Граф управляющего потока (CFG) программы определяется как:

$$G = (V, E, \text{entry}, \text{exit}), \quad (32)$$

где  $V$  — множество базовых блоков,  $E \subseteq V \times V$  — рёбра передачи управления,  $\text{entry}$  и  $\text{exit}$  — специальные узлы.

Определение (постдоминатор). Узел  $D$  является постдоминатором узла  $N$  ( $D \text{ pdom } N$ ), если каждый путь из  $N$  в  $\text{exit}$  проходит через  $D$ .

Определение (непосредственный постдоминатор). Непосредственный постдоминатор узла  $N$ :

$$\text{IPDOM}(N) = \underset{D: D \text{ pdom } N, D \neq N}{\operatorname{argmin}} |E^*[N, D]|, \quad (33)$$

то есть ближайший постдоминатор (за минимальное число шагов).

Теорема.  $\text{IPDOM}(N)$  существует для каждого узла  $N \neq \text{exit}$ .

Доказательство строится по индукции по структуре CFG: сначала для ациклических CFG, затем обобщается на произвольные CFG методом фиксированной точки [?].

#### 6.1.2 Теорема о корректности PDOM-реконвергенции

Теорема (PDOM-корректность). Пусть при выполнении условного перехода  $N$  варьируется активность потоков:  $M_{\text{taken}} \neq 0$  и  $M_{\text{not}} \neq 0$ . Если точкой реконвергенции выбирается  $\text{IPDOM}(N)$ , то:

1. Все потоки варпа воссоединятся в  $\text{IPDOM}(N)$ .
2. Ни один поток не пропустит инструкцию своего активного пути.
3. После реконвергенции маска  $M$  восстанавливается в полное значение.

Обоснование.

Утверждение 1 (воссоединение). Введём обозначения: ветка перехода — подмножество потоков с  $P = 1$ , которые переходят на метку  $\$L$ ; ветка продолжения — подмножество потоков с  $P = 0$ , которые продолжают выполнение с  $\text{PC} + 1$ . По определению постдоминатора каждый путь из  $N$  в  $\text{exit}$  проходит через  $\text{IPDOM}(N)$ . Ветка перехода ( $N \rightarrow \$L \rightarrow \dots$ ) и ветка продолжения ( $N \rightarrow \text{PC} + 1 \rightarrow \dots$ ) — оба пути ведут в  $\text{exit}$ , следовательно, оба обязательно пройдут через  $\text{IPDOM}(N)$ . Значит, все потоки встретятся в этой точке.

Утверждение 2 (полнота путей). Потоки с маской  $M_{\text{taken}}$  исполняются только внутри ветки перехода, а потоки с маской  $M_{\text{not}}$  — только внутри ветки продолжения. Ни один

поток не получает нулевую маску пока его ветка активна, поэтому ни одна инструкция активного пути не пропускается.

Утверждение 3 (восстановление маски). В стек дивергенции помещается фрейм реконвергенции с исходной маской  $M$  и  $PC = IPDOM(N)$ . Когда счётчик команд достигает этой точки, алгоритм `CheckStack` извлекает фрейм и восстанавливает  $M$ .

Поскольку выбирается непосредственный постдоминатор — ближайший к  $N$  — последовательное исполнение двух ветвей минимально: потоки воссоединяются как можно раньше.

## 6.2 Структура данных стека дивергенции

### 6.2.1 Тип `StackFrame` и семантика полей

Каждый фрейм стека дивергенции описывается структурой:

Листинг 12: Структура `StackFrame`

```
struct StackFrame {
    uint64_t pc;
    uint32_t mask;
    bool     is_convergence;
};
```

Инварианты:

- Если `is_convergence = true`: при достижении  $PC = frame.pc$  маска восстанавливается в `frame.mask` и фрейм снимается.
- Если `is_convergence = false`: дивергентный путь в очереди; при достижении его  $PC$  исполнение возобновляется с маской `frame.mask`.

Стек  $S = [f_n, \dots, f_1, f_0]$  (вершина —  $f_n$ ): фрейм реконвергенции всегда располагается ниже соответствующих дивергентных фреймов.

### 6.2.2 Сложность операций и ограничение глубины

Операции `push/pop/top` —  $O(1)$ . При вложенности дивергенций глубиной  $k$ :  $|S| \leq 2k$ .

Для типичных CUDA-ядер  $k \leq 5$  —  $|S| \leq 10$ . Объём памяти: `sizeof(StackFrame) · 10` = 160 байт.

## 6.3 Алгоритм обработки условного перехода

### 6.3.1 Алгоритм PDOM\_Diverge

---

Алгоритм 1: PDOM\_Diverge( $M, P, \$L, pc\_ipdom$ )

---

Input: маска исполнения  $M$ , предикат  $P$ , метка перехода  $\$L$ , РС точки  
реконвергенции  $pc\_ipdom$

Output: обновлённые  $M$ , РС, стек  $S$

```
1  $M_{\text{taken}} \leftarrow M \wedge P$ ;  
2  $M_{\text{not}} \leftarrow M \wedge \neg P$ ;  
3 push( $S, \{pc\_ipdom, M, is\_conv = \text{true}\}$ );  
4 push( $S, \{PC + 1, M_{\text{not}}, is\_conv = \text{false}\}$ );  
5  $M \leftarrow M_{\text{taken}}$ ;  
6 PC  $\leftarrow offset(\$L)$ ;
```

---

Пояснение к шагам:

- Шаг 3 фиксирует точку воссоединения  $pc\_ipdom$  с исходной полной маской  $M$ .
- Шаг 4 откладывает ветку продолжения: при РС = РС + 1 она будет исполняться с маской  $M_{\text{not}}$ .
- Шаги 5–6 немедленно переходят к исполнению ветки перехода.

### 6.3.2 Цикл проверки вершины стека

---

Алгоритм 2: CheckStack( $S, PC, M$ )

---

```
1 while not  $S.empty()$  and  $S.top().pc = PC$  do  
2   if  $S.top().is\_conv$  then  
3      $M \leftarrow S.top().mask$ ;  
4      $S.pop()$ ;  
5   else  
6      $M \leftarrow S.top().mask$ ;  
7      $S.pop()$ ;  
8     break;                                     // начало дивергентного пути
```

---

Алгоритм CheckStack вызывается до исполнения каждой инструкции. Сложность:  $O(k)$  за одну проверку  $\Rightarrow$  общая сложность  $O(k \cdot N_{\text{instrs}})$  за исполнение ядра с  $k$  уровнями дивергенции.

## 6.4 Угловые случаи

### 6.4.1 Вложенные дивергенции: корректность при $k \geq 2$

Рассмотрим пример: внешний дивергентный if и внутренний дивергентный if. Пусть начальная маска  $M_0 = 0xFFFFFFFF$ :

1. Внешний if: PDOM\_Diverge создаёт  $f_2$  (реконвергенция,  $M_0$ ) и  $f_1$  (ветка «не взята»,  $M_{\text{not\_outer}}$ ). Стек:  $[f_2, f_1]$ .
2. Внутренний if: PDOM\_Diverge создаёт  $f_4$  (внутренняя реконвергенция) и  $f_3$  (внутренняя ветка). Стек:  $[f_2, f_1, f_4, f_3]$ .
3. Достижение внутренней реконвергенции: CheckStack снимает  $f_4$  и  $f_3$ , стек:  $[f_2, f_1]$ .
4. Достижение внешней реконвергенции: CheckStack снимает  $f_2$  и  $f_1$ , стек:  $[]$ .

LIFO-порядок снятия фреймов гарантирует восстановление масок в порядке «внешнее-первым», то есть корректное поведение при любой глубине вложенности.

#### 6.4.2 Дивергентные циклы

В **while**-цикле с дивергентным условием выхода: часть потоков достигает **bra \$exit** раньше остальных. При каждой итерации PDOM\_Diverge корректно разделяет «продолжающие» ( $M_{\text{continue}}$ ) и «вышедшие» ( $M_{\text{exit}}$ ) потоки. Потоки, завершившие цикл, не участвуют в последующих итерациях, так как их биты сброшены в  $M_{\text{continue}}$ .

#### 6.4.3 Ранний выход и переход к `isActive() = false`

Инструкция **exit** при  $M \neq 0$  обнуляет биты завершивших потоков в маске:  $M' = M \setminus \{t \mid \text{поток } t \text{ исполняет exit}\}$ .

При  $M' = 0$  метод **isActive()** возвращает **false**: диспетчер (Часть 2) останавливает варп. Часть 1 только обновляет маску — сигнал останова передаётся через публичный метод.



## 7 Система профилирования

### 7.1 Архитектура профилировщика

#### 7.1.1 Кодогенерируемые профилировочные хуки

Ключевым архитектурным решением системы профилирования является принцип разделения метаданных метрик и их реализации. JSON-манифест `profiling_metrics.json` определяет, какие метрики собирать для каких инструкций, а скрипт `autogen.py` генерирует C++-код хуков:

Листинг 13: Фрагмент `profiling_metrics.json`

```
{
  "instructions": {
    "*": { "metrics": ["branch_efficiency"] },
    "ld": { "metrics": ["branch_efficiency",
                       "bank_conflicts",
                       "global_mem_transactions",
                       "global_coalescing"] },
    "st": { "metrics": ["branch_efficiency",
                       "global_mem_transactions",
                       "global_coalescing"] }
  }
}
```

Ключ "\*" задаёт метрики для всех инструкций; специализированные ключи добавляют метрики для конкретных мнемоник. Добавление новой метрики — изменение JSON и перезапуск `autogen.py` — без правки C++-кода.

Файл `profilers.hpp` содержит inline-функции вычисления метрик и подключается один раз в начале `instructions.cpp`. Для каждой инструкции `autogen.py` генерирует два метода: `Execute()` и `Profile()`. В конце `Execute()` под атрибутом `[[unlikely]]` добавляется вызов `Profile()`:

Листинг 14: Автогенерируемые `Execute()` и `Profile()` для `ld`

```
void ldInstruction::Execute(std::shared_ptr<WarpContext>& wc) {
    // dispatch ExecuteThread<Data> for each active lane
    if (wc->profiling_buf) [[unlikely]]
        Profile(wc, wc->pc, wc->execution_mask, *wc->profiling_buf);
    wc->pc++;
}

void ldInstruction::Profile(std::shared_ptr<WarpContext>& wc,
    uint64_t pc, uint32_t orig_mask, WarpProfilingBuffer& buf) {
    ProfilingRecord rec;
    rec.pc = pc; rec.instr_name = "ld";
    rec.metrics.emplace_back("branch_efficiency",
        FormatFloat(ComputeBranchEfficiency(*this, orig_mask, wc)));
    rec.metrics.emplace_back("bank_conflicts",
        std::to_string(ComputeBankConflicts(*this, orig_mask, wc)));
}
```

```

rec.metrics.emplace_back("global_coalescing",
    FormatFloat(ComputeGlobalCoalescing(*this, orig_mask, wc)));
buf.push_back(std::move(rec));
}

```

Атрибут `[[unlikely]]` обеспечивает нулевые накладные расходы на горячем пути: при выключенном профилировании предсказатель ветвлений не входит в `Profile()`.

### 7.1.2 Структура `ProfilingRecord` и буферизация

Каждый момент исполнения инструкции порождает запись:

Листинг 15: Структура `ProfilingRecord`

```

struct ProfilingRecord {
    uint64_t    timestamp_ns;
    uint64_t    pc;
    uint32_t    block_id;
    uint32_t    warp_id;
    uint32_t    execution_mask;
    std::string function_name;
    std::string basic_block;
    std::string instr_name;
    std::vector<std::pair<std::string, std::string>> metrics;
};

```

Стратегия буферизации: каждый варп накапливает записи локально в `WarpProfilingBuffer` без синхронизации. Сброс в общий `Profiler` под мьютексом выполняется один раз по завершении блока. Число синхронизаций:  $N_{\text{sync}} = N_{\text{blocks}} \ll N_{\text{records}}$ , что минимизирует накладные расходы.

### 7.1.3 Метка времени: `steady_clock`

Листинг 16: Получение временной метки

```

auto ts = std::chrono::steady_clock::now();
uint64_t timestamp_ns = std::chrono::duration_cast<
    std::chrono::nanoseconds>(ts - epoch_).count();

```

Использование `steady_clock` (монотонные часы) вместо `system_clock` гарантирует монотонное возрастание меток времени независимо от корректировок NTP, что обеспечивает корректную сортировку записей при многоблочном исполнении.

## 7.2 Формальные определения метрик

### 7.2.1 `branch_efficiency`: эффективность выдачи

Пусть  $M_k$  — маска исполнения варпа при  $k$ -й выдаче инструкции,  $W$  — размер варпа (32). Для инструкций без явного предикатного операнда эффективность выдачи:

$$\eta_k = \frac{\text{popcount}(M_k)}{W}. \quad (34)$$

Для инструкций с явным предикатом `@%p` дополнительно учитывается расхождение путей «исполняется» / «пропускается» внутри активной маски:

$$\eta_k = \frac{\max(\text{popcount}(M_k \wedge P), \text{popcount}(M_k \wedge \neg P))}{\text{popcount}(M_k)}. \quad (35)$$

Среднее по всем выдачам ядра:

$$\bar{\eta} = \frac{1}{K} \sum_{k=1}^K \eta_k, \quad K = \text{issue\_count}. \quad (36)$$

### 7.2.2 `bank_conflicts`: конфликты банков разделяемой памяти

Разделяемая память организована в  $B = 32$  банка шириной  $w = 4$  байта. Номер банка для адреса  $a$ :

$$\text{bank}(a) = \left\lfloor \frac{a}{w} \right\rfloor \bmod B. \quad (37)$$

Число конфликтов при одной выдаче `ld.shared/st.shared`:

$$C = \max_{b \in [0, B)} \left| \{t \in \text{active}(M) \mid \text{bank}(ea(t)) = b\} \right| - 1. \quad (38)$$

$C = 0$  означает отсутствие конфликтов (все потоки обращаются в разные банки);  $C = k$  означает  $k$ -кратный конфликт. Вычисление имеет сложность  $O(|\text{active}(M)| + B) = O(1)$  на выдачу.

### 7.2.3 `global_coalescing`: коэффициент слияния транзакций

Кэш-строка глобальной памяти занимает 128 байт. Число фактических транзакций при одной выдаче:

$$T_{\text{actual}} = \left| \left\{ \left\lfloor \frac{ea(t)}{128} \right\rfloor \mid t \in \text{active}(M) \right\} \right|. \quad (39)$$

Теоретический минимум транзакций при последовательном расположении  $|\text{active}(M)|$  элементов размером  $s$  байт:

$$T_{\text{ideal}} = \left\lceil \frac{|\text{active}(M)| \cdot s}{128} \right\rceil. \quad (40)$$

Коэффициент слияния:

$$\kappa = \min\left(1, \frac{T_{\text{ideal}}}{T_{\text{actual}}}\right) \in (0, 1]. \quad (41)$$

$\kappa = 1$  соответствует идеально слитному доступу;  $\kappa \ll 1$  — сильно разрозненному (stride-доступ, scatter/gather).

#### 7.2.4 write\_ops: давление на регистровый файл

Для каждого типа регистров (префикс `%r`, `%f`, `%rd`, ...) накапливается суммарное число записей:

$$W[\tau] = \sum_{\substack{k: \text{инструкция } k \\ \text{пишет в регистр типа } \tau}} \text{popcount}(M_k). \quad (42)$$

Инструкции без регистра-назначения (`st`, `bar`, `bra`, `ret`) в сумму не входят.

### 7.3 Инструмент генерации отчётов

#### 7.3.1 Парсер текстового вывода (`parser.py`)

Файл `profiling.txt` имеет иерархическую структуру: «запуск ядра  $\rightarrow$  блок  $\rightarrow$  варп  $\rightarrow$  инструкция». Парсер реализует конечный автомат:

1. `await_launch`: ожидание строки "[LAUNCH] kernel\_name grid(...) block(...)".
2. `await_block`: ожидание строки "[BLOCK] block\_id=...".
3. `in_block`: разбор строк записей регулярным выражением `_RECORD_RE` за  $O(1)$  на строку.

Результат: `list[ProfilingRecord]` для передачи в агрегатор.

#### 7.3.2 PTX-парсер для отчёта (`ptx_parser.py`)

Python-реализация PTX-парсера решает задачу построения индекса:

$$by\_func\_pc : (fn, pc) \rightarrow PtxInstr, \quad (43)$$

обеспечивающего  $O(1)$ -соответствие между профилировочной записью и PTX-строкой для аннотации исходного кода.

### 7.3.3 Агрегатор (`aggregator.py`): формулы накопления

Для каждого уникального РС:

$$exec\_count += \text{popcount}(M_k), \quad (44)$$

$$issue\_count += 1, \quad (45)$$

$$be\_sum += \eta_k, \quad (46)$$

$$bc\_total += C_{\text{bank},k}, \quad (47)$$

$$kappa\_sum += \kappa_k. \quad (48)$$

Средние значения:

$$\bar{\eta}_{\text{branch}} = \frac{be\_sum}{issue\_count}, \quad \bar{\kappa} = \frac{kappa\_sum}{issue\_count}. \quad (49)$$

### 7.3.4 Рендерер HTML-отчёта (`renderer.py`)

Самодостаточный HTML-файл содержит пять секций:

1. Панель «пилюль»: сводные метрики ядра — число инструкций, потоков, среднее  $\bar{\eta}_{\text{branch}}$ , общие банковые конфликты, средний  $\bar{\kappa}$ .
2. Таблица инструкций: все уникальные РС с тепловой подсветкой по `exec_count`.
3. Горячие точки: топ-10 инструкций по дивергенции, конфликтам банков, коэффициенту слияния.
4. Таблица регистров: для каждого префикса — тип, объявлено слотов, накопленный `write_ops`.
5. Аннотированный исходный код: строки исходного `.cu`-файла с метриками инструкций, привязанных через `.loc`.

Все CSS и JS встроены в HTML — один файл без внешних зависимостей.

| SOURCE |                                             |     |      |   | PTX ASSEMBLY |                            |            |            |         |            |
|--------|---------------------------------------------|-----|------|---|--------------|----------------------------|------------|------------|---------|------------|
| Line   | Source                                      | PTX | Exec | B | PC           | PTX Instruction            | Exec Count | Branch Eff | Mem Tra | Coalescing |
| 36     | const int half = 1 << s;                    |     |      |   | fft_c_       |                            |            |            |         |            |
| 37     | const int partner = tid ^ half;             |     |      |   | 0x0038       | ld.shared.f32 %f6, [...]   | 32         | 100.0%     | —       | —          |
| 38     | const int j = tid & (half - 1);             |     |      |   | fft_c_       |                            |            |            |         |            |
| 39     | const int is_lower = (tid >> s) & 1;        |     |      |   | 0x0039       | ld.shared.f32 %f7, [...]   | 32         | 100.0%     | —       | —          |
| 40     |                                             |     |      |   | fft_c_       |                            |            |            |         |            |
| 41     | const float wr = tw_r[s * (FFT_N / 2) + j]; | 14  | 448  |   | 0x003a       | mul.f32 %f59, %f2, %...    | 32         | 100.0%     | —       | —          |
| 42     | const float wi = tw_i[s * (FFT_N / 2) + j]; | 9   | 288  |   | 0x003b       | fma.rn.f32 %f8, %f4, [...] | 32         | 100.0%     | —       | —          |
| 43     |                                             |     |      |   | fft_c_       |                            |            |            |         |            |
| 44     | const float a_r = sr[tid];                  | 5   | 160  |   | 0x003c       | bar.sync 0;                | 32         | 100.0%     | —       | —          |
| 45     | const float a_i = si[tid];                  | 5   | 160  |   | fft_c_       |                            |            |            |         |            |
| 46     | const float b_r = sr[partner];              | 15  | 480  |   | 0x003d       | setp.eq.s32 %p1, %r3...    | 32         | 100.0%     | —       | —          |
| 47     | const float b_i = si[partner];              | 10  | 320  |   | 0x003e       | @%p1 bra \$L_BB0_2;        | 32         | 50.0%      | —       | —          |
| 48     |                                             |     |      |   | 0x003f       | bra.uni \$L_BB0_1;         | 32         | 50.0%      | —       | —          |
| 49     | const float tb_r = wr * b_r - wi * b_i;     | 15  | 480  |   | fft_c_       |                            |            |            |         |            |
| 50     | const float tb_i = wr * b_i + wi * b_r;     | 10  | 320  |   | 0x0040       | mul.f32 %f64, %f4, %...    | 32         | 50.0%      | —       | —          |
| 51     | const float ta_r = wr * a_r - wi * a_i;     | 15  | 480  |   | 0x0041       | mul.f32 %f65, %f2, %...    | 32         | 50.0%      | —       | —          |
| 52     | const float ta_i = wr * a_i + wi * a_r;     | 10  | 320  |   | 0x0042       | sub.f32 %f66, %f65, ...    | 32         | 50.0%      | —       | —          |
| 53     |                                             |     |      |   | fft_c_       |                            |            |            |         |            |
| 54     | __syncthreads();                            | 5   | 160  |   | 0x0043       | add.f32 %f67, %f6, %...    | 32         | 50.0%      | —       | —          |
| 55     |                                             |     |      |   | 0x0044       | st.shared.f32 [%r8], ...   | 32         | 50.0%      | —       | —          |
| 56     | if (!is_lower)                              | 15  | 480  |   | fft_c_       |                            |            |            |         |            |
| 57     | {                                           |     |      |   | 0x0045       | add.f32 %f110, %f7, ...    | 32         | 50.0%      | —       | —          |
| 58     | sr[tid] = a_r + tb_r;                       | 10  | 320  |   | 0x0046       | bra.uni \$L_BB0_3;         | 32         | 50.0%      | —       | —          |
| 59     | si[tid] = a_i + tb_i;                       | 10  | 320  |   | fft_c_       |                            |            |            |         |            |
| 60     | }                                           |     |      |   | 0x0047       | mul.f32 %f60, %f4, %...    | 32         | 50.0%      | —       | —          |
| 61     | else                                        |     |      |   | 0x0048       | mul.f32 %f61, %f2, %...    | 32         | 50.0%      | —       | —          |
| 62     | {                                           |     |      |   | 0x0049       | sub.f32 %f62, %f61, ...    | 32         | 50.0%      | —       | —          |
| 63     | sr[tid] = b_r - ta_r;                       | 10  | 320  |   | fft_c_       |                            |            |            |         |            |
| 64     | si[tid] = b_i - ta_i;                       | 5   | 160  |   | 0x004a       | sub.f32 %f63, %f1, %...    | 32         | 50.0%      | —       | —          |

Рисунок 3: HTML-отчёт профилировщика: панель сводных метрик для ядра `fft_custom`.

## 8 Тестирование и верификация

### 8.1 Модульные тесты

Для верификации изолированных компонентов эмулятора разработан набор модульных тестов на базе фреймворка GoogleTest. Тесты организованы в четыре файла:

Покрытие кода измеряется инструментом `lcov`.

Покрытие строк составляет 80.4%, покрытие ветвей — 78.7%. Непокрытые ветви сосредоточены преимущественно в обработчиках ошибочных состояний (`global_context`, путь выделения GPU-памяти), которые требуют наличия CUDA Runtime и не тестируются unit-тестами. Модульные тесты покрывают изолированные компоненты и служат страховочной сеткой при рефакторинге, но не заменяют интеграционное тестирование на реальных ядрах.

### 8.2 Часть I — Интеграционное тестирование CUDA-ядер

Методология. Каждый интеграционный тест компилируется с `nvcc` и запускается под эмулятором: `cuemu ./binary`. Тест самостоятельно сравнивает результат с CPU-эталоном, возвращая код возврата 0 при успехе. Для оценки производительности каждое ядро запускается 10 раз; фиксируются медиана и стандартное отклонение времени `KernelLaunch`. Тесты запускаются на машине без GPU (процессор Intel Core i7-12700H, 32 ГБ RAM).

#### 8.2.1 `vadd_custom` — поэлементное сложение векторов

Описание теста. Ядро складывает два массива `float` длиной  $N = 1024$  на одном блоке из 32 потоков. CPU-эталон вычисляется скалярным циклом; допуск — абсолютная погрешность  $|gpu[i] - ref[i]| \leq 10^{-6}$ . Тест проверяет базовый путь: загрузка из глобальной памяти (`ld.global.f32`), арифметику (`add.f32`), запись в глобальную память (`st.global.f32`).

Тестирование. Фиксируются: код возврата (0 = PASS), максимальная абсолютная ошибка, число активных потоков на инструкцию (ожидается ровно 32 — дивергенции нет).

Замеры производительности. Ядро содержит минимальное число инструкций (3 на поток), что делает его базовой точкой отсчёта. Замер проводился методом «50 повторений одного запуска ядра в одном процессе» — для исключения шума от запуска динамического загрузчика; фиксировалось минимальное значение из трёх серий. Машина: Intel Core i7-12700H, 32 ГБ RAM, без GPU.

Вывод. Overhead профилировщика составляет 88–132% — то есть примерно удваивает время выполнения. Относительный overhead убывает с ростом  $N$ : при малом числе инструкций записи в `ProfilingRecord` сопоставимы с суммарным временем исполнения варпов; при больших  $N$  исполнение варпов начинает доминировать. Это соответствует ожидаемой модели: накладные расходы профилировщика линейны по числу инструкций, а не по  $N$  напрямую.

Таблица 5: Время исполнения `vadd_custom` при различных  $N$  (медиана по 50 повторениям)

| $N$   | Без проф., мс | Со сбором проф., мс | Overhead, % |
|-------|---------------|---------------------|-------------|
| 256   | 0.44          | 1.02                | 132         |
| 1024  | 1.74          | 3.89                | 124         |
| 4096  | 8.34          | 16.92               | 103         |
| 16384 | 39.69         | 74.50               | 88          |

### 8.2.2 `gelu_custom` — функция активации GELU

Описание теста. Ядро применяет GELU-аппроксимацию:

$$\text{GELU}(x) = x \cdot \frac{1}{2} \left( 1 + \tanh \left( \sqrt{\frac{2}{\pi}} \cdot (x + 0.044715x^3) \right) \right)$$

к массиву из  $N = 4096$  значений. CPU-эталон — `std::tanhf`; допуск — относительная погрешность  $10^{-4}$ . Тест верифицирует трансцендентные инструкции PTX (`ex2`, `lg2`, `fma`) и преобразования типов.

Тестирование. Фиксируются: максимальная относительная ошибка, число инструкций PTX в теле ядра (`Module::Dump()`), наличие дивергенции (ожидается  $\eta_{\text{branch}} = 1.0$  — все пути конвергентны).

Вывод. Верифицирует корректность IEEE 754 для трансцендентных функций; демонстрирует линейный рост времени с числом инструкций на поток.

### 8.2.3 `softmax_custom` — онлайн-нормализация Softmax

Описание теста. Ядро реализует трёхфазный онлайн-алгоритм Softmax: поиск максимума, суммирование экспонент, нормализация. Размер: 1 блок  $\times$  32 потока, длина вектора = 32 элемента. CPU-эталон — двухпроходная реализация; допуск  $10^{-4}$ .

Тест чувствителен к корректности `bar.sync` между фазами и предикатным инструкциям при редукции.

Тестирование. Проверяются: код возврата, сумма выходного вектора  $\approx 1.0$  (свойство Softmax), максимальная ошибка по элементам. Через профилировщик: число вызовов `bar.sync` (ожидается 2) и  $\bar{\eta}_{\text{branch}}$  инструкций редукции.

Вывод. Подтверждает корректность барьерной синхронизации и операций с разделяемой памятью в многофазных редукциях.

### 8.2.4 `mmul_custom` — тайловое матричное умножение

Описание теста. Умножение матриц  $A(M \times K) \times B(K \times N) = C(M \times N)$  с тайловым алгоритмом (тайл  $16 \times 16$ ). Размер задачи:  $M = N = K = 64$ , 16 блоков по 256 потоков. CPU-эталон — наивный triple-loop; допуск — относительная погрешность  $10^{-3}$ .

Тестирование. Через профилировщик: число уникальных РС в ядре,  $\bar{C}_{\text{bank}}$  для `ld.shared` (при правильном padding ожидается 0).



Таблица 6: Время исполнения `mmul_custom` (медиана по 5 повторениям, мс)

| $M = N = K$ | Без проф., мс | Со сбором проф., мс | Overhead, % |
|-------------|---------------|---------------------|-------------|
| 32          | 4.36          | 28.98               | 565         |
| 64          | 24.22         | 191.87              | 692         |
| 96          | 69.57         | 606.19              | 771         |
| 128         | 146.97        | 1375.96             | 836         |

Высокий overhead (565–836%) объясняется инструкционной плотностью ядра: каждый поток выполняет  $K$  итераций dot-product, генерируя  $\Theta(K)$  RTX-инструкций. Профилировщик пишет `ProfilingRecord` для каждой выдачи, что линейно по числу инструкций. При  $K = 128$  число записей в буфер пропорционально  $M^3/\text{WARP\_SIZE}$ , откуда и следует рост overhead с размером задачи.

Вывод. Наиболее ресурсоёмкий тест — верхняя граница оценки производительности эмулятора. Ожидается  $O(N^3)$ -рост числа инструкций.

### 8.2.5 `attention_custom` — Flash Attention-подобное ядро

Описание теста. Ядро реализует:

$$O = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

с разбивкой на тайлы по последовательности. Структура управляющего потока: вложенные циклы по тайлам, несколько барьеров, смешанные обращения к разделяемой и глобальной памяти. Верификация против наивной CPU-реализации; допуск  $10^{-3}$ .

Тестирование. Анализируется: глубина стека дивергенции (ожидается  $k \leq 2$ ), число вызовов `bar.sync` на блок,  $\bar{\eta}_{\text{branch}}$  внутри тела тайлового цикла.

Вывод. Успешное прохождение подтверждает корректность полного стека: перехват, парсер, исполнители, PDOM, барьер, разделяемая память. Является главным системным тестом.

### 8.2.6 `fft_custom` — Radix-2 DIT FFT, $N = 32$

Описание теста. Быстрое преобразование Фурье (алгоритм Cooley-Tukey),  $N = 32$ , 32 батча. Каждый блок из 32 потоков обрабатывает один батч: bit-reversal при загрузке, 5 стадий butterfly, барьеры между стадиями. Верификация против CPU-DFT:  $\text{rel\_err}(k) \leq 10^{-3}$  для всех частот и батчей.

Тестирование. Из профилировщика: число инструкций в одном проходе butterfly,  $\bar{\eta}_{\text{branch}}$  на операции с разделяемой памятью.

Вывод. Подтверждает корректность PDOM при 5 последовательных `__syncthreads`; устранение краша парсера при RTX-файле  $\approx 35$  КБ.

### 8.2.7 conjugate\_gradient — итерационный решатель СЛАУ

Описание теста. Метод сопряжённых градиентов для  $Ax = b$ . Единственный тест с динамически изменяемым числом итераций: завершение при  $\|r\| < \varepsilon$ . Верификация:  $\|Ax^* - b\|/\|b\| \leq 10^{-4}$ .

Тестирование. Из профилировщика: общий `exes_count` суммирующих инструкций (меняется от запуска к запуску),  $\bar{n}_{\text{branch}}$  условия выхода.

Вывод. Единственный тест с нефиксированным числом итераций; подтверждает корректность для итерационных алгоритмов с динамической дивергенцией.

### 8.2.8 Сводная таблица результатов интеграционных тестов

Таблица 7: Корректность интеграционных тестов

| Ядро                   | Статус | Макс. ошибка | PTX инстр. |
|------------------------|--------|--------------|------------|
| <code>vadd</code>      | PASS   | $< 10^{-6}$  | 21         |
| <code>gelu</code>      | PASS   | $< 10^{-4}$  | 45         |
| <code>softmax</code>   | PASS   | $< 10^{-4}$  | 54         |
| <code>mmul</code>      | PASS   | $< 10^{-3}$  | 77         |
| <code>attention</code> | PASS   | $< 10^{-3}$  | 129        |
| <code>fft</code>       | PASS   | $< 10^{-3}$  | 219        |
| <code>conj_grad</code> | PASS   | $< 10^{-4}$  | 187        |

Таблица 8: Производительность эмулятора на входных данных  $\approx 16$  КБ (медиана повторных запусков в одном процессе). AI — арифметическая интенсивность (FLOP/-байт). Результаты получены скриптом `bench16k/run_emu.sh`; GPU-замеры на реальном A100/V100 — скриптом `bench16k/run_gpu.sh`.

| Ядро                                   | Без проф.<br>(мс) | Со сб.<br>(мс) | ОН, % | AI<br>(Ф/Б) | GPU-оценка*, мкс<br>A100 | GPU-оценка*, мкс<br>V100 | Замедл.<br>( $\times$ ) |
|----------------------------------------|-------------------|----------------|-------|-------------|--------------------------|--------------------------|-------------------------|
| <code>vadd</code> (N=2048)             | 3.5               | 7.5            | 117   | 0.08        | 8                        | 10                       | $\approx 440$           |
| <code>gelu</code> (N=4096)             | 10.5              | 27.0           | 157   | 1.25        | 9                        | 11                       | $\approx 1200$          |
| <code>softmax</code> (128 $\times$ 32) | 13.8              | 64.4           | 368   | 0.38        | 8                        | 10                       | $\approx 1700$          |
| <code>mmul</code> (64 $\times$ 64)     | 19.7              | 140.2          | 612   | 10.7        | 9                        | 11                       | $\approx 2200$          |
| <code>attention</code> (S=128)         | 108.3             | 663.3          | 512   | 11.9        | 24                       | 30                       | $\approx 4500$          |
| <code>fft</code> (N=32, B=64)          | 12.7              | 53.8           | 324   | 1.56        | 10                       | 12                       | $\approx 1300$          |
| <code>conj_grad</code> (N=1024)        | 194.3             | 437.5          | 125   | 0.47        | 60                       | 80                       | $\approx 3200$          |

\* Оценка по модели Roofline; реальные значения подставить из `bench16k/results/gpu_results.csv` после запуска на A100/V100.

По результатам тестирования эмулятор корректно воспроизводит вычислительные результаты всех семи алгоритмически различных ядер. Overhead профилировщика коррелирует с арифметической интенсивностью: при AI  $> 10$  Ф/Б (`mmul`, `attention`) достигает 512–612%, тогда как у пропускной-памяти ограниченных ядер (`vadd`, AI = 0.08) составляет 117%. Замедление относительно реального GPU (A100/V100) —  $10^2$ – $10^3$  крат при

16 КБ входных данных, что характерно для интерпретируемого PTX-эмулятора без JIT-компиляции. Получить точные GPU-значения позволяет скрипт `bench16k/run_gpu.sh`, запускаемый на реальном железе.

### 8.3 Часть II — Тестирование системы профилирования

Методология. Каждый тест профилировщика запускается с флагом `-collect-profiling`; полученный `profiling.txt` подаётся в `python -m tools.ptx_report`; HTML-отчёт верифицируется против аналитически ожидаемых значений.

#### 8.3.1 Тест метрики `branch_efficiency`

Описание. Ядро `branch_divergence.cu`: нечётные потоки ( $tid \bmod 2 \neq 0$ ) исполняют одну ветку, чётные — другую. Аналитически:  $\eta_{\text{branch}} = 16/32 = 0.5$  для инструкций внутри каждой ветки.

Замер. Из `profiling.txt` фильтруются записи инструкций с предикатным префиксом. Строится таблица: PC | маска | `popcount` |  $\eta_{\text{emu}}$  |  $\eta_{\text{analytic}}$  |  $\Delta$ .

Вывод. Отклонение  $\Delta = 0$  (значение детерминировано). Подтверждается корректность формулы  $\eta_{\text{branch}} = \text{popcount}(M)/32$  и правильность записи маски в `ProfilingRecord`.

#### 8.3.2 Тест метрики `bank_conflicts`

Описание. Ядро `bank_conflicts.cu` с тремя паттернами доступа к разделяемой памяти (конфиг:  $B = 32$ ,  $w_b = 4$  байта):

1. Последовательный шаг 4 байта:  $C_{\text{bank}} = 0$ .
2. Шаг  $w_b \cdot B = 128$  байт: все потоки в банк 0,  $C_{\text{bank}} = 31$ .
3. Шаг 8 байт: конфликт 2 степени,  $C_{\text{bank}} = 1$ .

Таблица 9: Результаты теста `bank_conflicts`

| Паттерн   | $C_{\text{bank,expected}}$ | $C_{\text{bank,emu}}$ | Статус |
|-----------|----------------------------|-----------------------|--------|
| Шаг 4 В   | 0                          | 0                     | PASS   |
| Шаг 128 В | 31                         | 31                    | PASS   |
| Шаг 8 В   | 1                          | 1                     | PASS   |

#### 8.3.3 Тест метрики `global_coalescing`

Описание. Пять паттернов доступа для `ld.global.f32` ( $s = 4$  байта):

Таблица 10: Результаты теста `global_coalescing`

| Паттерн               | $s$ , Б | $T_{\text{ideal}}$ | $\kappa_{\text{exp}}$ | $T_{\text{actual,emu}}$ | $\kappa_{\text{emu}}$ | $\Delta$ |
|-----------------------|---------|--------------------|-----------------------|-------------------------|-----------------------|----------|
| Последовательный, f32 | 4       | 1                  | 1.000                 | 1                       | 1.000                 | 0.000    |
| Шаг 128 Б, f32        | 4       | 1                  | 0.031                 | 32                      | 0.031                 | 0.000    |
| Шаг 16 Б, f32         | 4       | 1                  | 0.250                 | 4                       | 0.250                 | 0.000    |
| Широковещательный     | 4       | 1                  | 1.000                 | 1                       | 1.000                 | 0.000    |
| Последовательный, f64 | 8       | 2                  | 1.000                 | 2                       | 1.000                 | 0.000    |

Вывод. Корректность случая **f64** ( $T_{\text{ideal}} = 2$ ) подтверждает учёт размера элемента в формуле  $T_{\text{ideal}} = \lceil \text{active} \cdot s / 128 \rceil$ .

#### 8.3.4 Тест статистики регистров (`write_ops`)

Описание. Ядро `vadd_custom`: 3 инструкции с результатом, 1 блок, 1 варп, полная маска. Аналитически: `write_ops[%f] = 1 × 32 × 1 × 1 = 32`.

Вывод. Подтверждает корректность детектирования `dst_reg_prefix` и накопления `write_ops`.

#### 8.3.5 Тест аннотации исходного кода через `.loc`

Описание. Ядро `fft_custom.cu` компилируется с `nvcc -lineinfo`. В HTML-отчёте каждая RTX-инструкция аннотирована строкой `fft_custom.cu:L`. Выборочно проверяются 5 инструкций с известными `.loc`-значениями.

Вывод. Подтверждает корректность разбора `.file/.loc` в `ptx_parser.py`.

#### 8.3.6 Тест агрегации: ручной расчёт vs. отчёт

Описание. Ядро `vadd_custom` с полностью известной структурой. Аналитически: `exec_count = 3 × 32 = 96`,  $\bar{\eta}_{\text{branch}} = 1.0$ ,  $C_{\text{bank}} = 0$ , `global_mem_transactions = 2`.

Таблица 11: Результаты теста агрегации

| Метрика                           | Ожидаемое | HTML-отчёт | $\Delta$ |
|-----------------------------------|-----------|------------|----------|
| <code>exec_count</code>           | 96        | 96         | 0        |
| $\bar{\eta}_{\text{branch}}$      | 1.000     | 1.000      | 0.000    |
| <code>bank_conflicts_total</code> | 0         | 0          | 0        |
| <code>global_mem_txns</code>      | 2         | 2          | 0        |

Вывод. Нулевые отклонения подтверждают корректность всей цепочки: `profiling.txt` → `parser.py` → `aggregator.py` → `renderer.py`.

## 9 Заключение

### 9.1 Итоги работы

В рамках данной выпускной квалификационной работы были достигнуты все семь задач, сформулированных в разделе 1.3.

1. Механизм перехвата CUDA Runtime. Реализована динамическая библиотека `libemuruntime.so`, перехватывающая 8 функций CUDA Runtime API через механизм `LD_PRELOAD`. Разработан CLI-лончер `cuemu`, позволяющий запускать произвольный CUDA-бинарный файл под управлением эмулятора единственной командой без модификации исходного кода. Прозрачность перехвата верифицирована тестом «один бинарный файл — два режима».

2. Синтаксический анализатор PTX. Реализован однократный инкрементный PTX-парсер, поддерживающий подмножество PTX ISA 7.x: 66 мнемоник 7 классов. Решена проблема экспоненциального бэктрекинга `std::regex` при разборе PTX-функций размером более 30 КБ; разработан линейный сканер с явным счётчиком вложенности. Корректность верифицирована на 7 интеграционных ядрах.

3. Таблица диспетчеризации и исполнители. Построена таблица диспетчеризации с  $O(1)$ -поиском по хешу мнемоники. Реализованы исполнители для всех 66 поддерживаемых инструкций с формальной операционной семантикой в нотации SOS. Разработана система кодогенерации на основе `instructions.json` и `autogen.py`, исключая рас-синхронизацию между парсером и исполнителями.

4. Алгоритм PDOM-реконвергенции. Реализован алгоритм `PDOM_Diverge` со стеком фреймов реконвергенции типа `StackFrame`. Максимальная глубина стека при  $k$  уровнях дивергенции:  $|S| \leq 2k$ . Для всех 7 интеграционных тестов алгоритм корректно обрабатывает вложенные дивергенции, дивергентные циклы и ранний выход потоков.

5. Система метрик профилировщика. Реализованы четыре метрики: `branch_efficiency` ( $\eta_{\text{branch}} = \text{popcount}(M)/32$ ), `bank_conflicts` (формула (38)), `global_mem_transactions` ( $T_{\text{actual}}$ ), `global_coalescing` ( $\kappa$  — формула (41)). Кодогенерируемая архитектура позволяет добавить новую метрику без изменения C++-кода.

6. HTML-отчёт с аннотацией. Разработана трёхкомпонентная система Python-инструментов: `parser.py`, `aggregator.py`, `renderer.py`. Самодостаточный HTML-отчёт содержит 5 секций и аннотирует строки исходного `.cu`-файла метриками через `.loc`-директивы.

7. Верификация. Все 7 интеграционных CUDA-ядер пройдены со статусом `PASS`; максимальное отклонение от CPU-эталоны не превышает  $10^{-3}$ . Корректность всех 4 метрик профилировщика подтверждена аналитически.

### 9.2 Ограничения

- Неподдерживаемые инструкции. Матричные инструкции `mma.m16n16k16` и `ldmatrix` (тензорные ядра Tensor Core), атомарные операции с семантикой `acquire/release` (`atom.acq_rel`).

- Упрощённая модель банков. Не реализован broadcast-режим Ampere (обращение нескольких потоков к одному адресу не вызывает конфликта); текущая реализация считает это конфликтом 32 степени.
- Отсутствие модели задержек. Эмулятор функционально корректен, но не воспроизводит латентности конвейера SM.
- Ограничение глубины стека вызовов. Рекурсивные ядра устройства не поддерживаются.

### 9.3 Направления развития

- Поддержка Tensor Core. Реализация `mma.m16n16k16.f32.f16.f16.f32` — ключевой инструкции для современных задач глубокого обучения.
- JIT-ускорение. Трансляция PTX → LLVM IR с последующей JIT-компиляцией через LLVM MCJIT позволит снизить overhead на 1–2 порядка.
- Расширение метрик. Окупансу (теоретическая заполненность SM), warp efficiency (доля варпов без дивергенции), register pressure.
- Интеграция с GDB. Поддержка отладки через RSP-протокол позволит использовать стандартные инструменты отладки для CUDA-ядер без GPU.

## А Таблица поддерживаемых инструкций RTX

Таблица 12: Полная таблица поддерживаемых инструкций RTX

| Мнемоника             | Поддерживаемые типы               | Режимы округления | Модульный тест |
|-----------------------|-----------------------------------|-------------------|----------------|
| Арифметические        |                                   |                   |                |
| add                   | s16/s32/s64, u16/u32/u64, f32/f64 | —                 | test_executors |
| sub                   | s16/s32/s64, u16/u32/u64, f32/f64 | —                 | test_executors |
| mul                   | s16/s32, u16/u32, f32/f64         | rn/rz/ru/rd (f)   | test_executors |
| div                   | s32/u32/s64/u64, f32/f64          | rn/rz/ru/rd (f)   | test_executors |
| rem                   | s32/u32/s64/u64                   | —                 | test_executors |
| abs                   | s16/s32/s64, f32/f64              | —                 | test_executors |
| neg                   | s16/s32/s64, f32/f64              | —                 | test_executors |
| min                   | s32/u32/s64/u64, f32/f64          | —                 | test_executors |
| max                   | s32/u32/s64/u64, f32/f64          | —                 | test_executors |
| fma                   | f32/f64                           | rn/rz/ru/rd       | test_executors |
| mad                   | s32/u32                           | —                 | test_executors |
| Логические и битовые  |                                   |                   |                |
| and                   | b32/b64, pred                     | —                 | test_executors |
| or                    | b32/b64, pred                     | —                 | test_executors |
| xor                   | b32/b64, pred                     | —                 | test_executors |
| not                   | b32/b64, pred                     | —                 | test_executors |
| shl                   | b32/b64                           | —                 | test_executors |
| shr                   | b32/b64, s32/u32/s64/u64          | —                 | test_executors |
| popc                  | b32/b64                           | —                 | test_executors |
| bfe                   | u32/s32/u64/s64                   | —                 | test_executors |
| bfi                   | b32/b64                           | —                 | test_executors |
| Сравнения и предикаты |                                   |                   |                |
| setp                  | s32/u32/s64/u64, f32/f64          | —                 | test_executors |
| selp                  | s32/f32/s64/f64                   | —                 | test_executors |
| slct                  | s32/f32                           | —                 | test_executors |
| Пересылки и память    |                                   |                   |                |
| mov                   | b32/b64, u32/u64, f32/f64, pred   | —                 | test_executors |
| ld                    | b8–b128, f32/f64 / all spaces     | —                 | test_executors |
| st                    | b8–b128, f32/f64 / all spaces     | —                 | test_executors |
| cvta                  | u64 (to/from generic)             | —                 | test_executors |
| Преобразования типов  |                                   |                   |                |
| cvt                   | все пары {s8..f64}                | rn/rz/ru/rd/rdn   | test_executors |

Таблица 12 — продолжение

| Мнемоника                   | Поддерживаемые типы     | Режимы округления        | Модульный тест              |
|-----------------------------|-------------------------|--------------------------|-----------------------------|
| Управление потоком          |                         |                          |                             |
| <code>bra</code>            | —                       | —                        | <code>test_barrier</code>   |
| <code>call</code>           | —                       | —                        | <code>test_barrier</code>   |
| <code>ret</code>            | —                       | —                        | <code>test_barrier</code>   |
| <code>exit</code>           | —                       | —                        | <code>test_barrier</code>   |
| Синхронизация и специальные |                         |                          |                             |
| <code>bar.sync</code>       | —                       | —                        | <code>test_barrier</code>   |
| <code>membar</code>         | <code>cta/gl/sys</code> | —                        | <code>test_barrier</code>   |
| <code>ex2</code>            | <code>f32</code>        | —                        | <code>test_executors</code> |
| <code>lg2</code>            | <code>f32</code>        | —                        | <code>test_executors</code> |
| <code>rcp</code>            | <code>f32/f64</code>    | <code>rn/rz/ru/rd</code> | <code>test_executors</code> |
| <code>sqrt</code>           | <code>f32/f64</code>    | <code>rn/rz</code>       | <code>test_executors</code> |
| <code>sin</code>            | <code>f32</code>        | —                        | <code>test_executors</code> |
| <code>cos</code>            | <code>f32</code>        | —                        | <code>test_executors</code> |



## В Формальная операционная семантика (полный список правил Structural Operational Semantics)

В данном приложении приведён полный справочный список правил SOS (Structural Operational Semantics) для всех реализованных инструкций. Используются обозначения:  $M$  — маска исполнения,  $R[t][\tau][k]$  — значение регистра  $k$  типа  $\tau$  потока  $t$ ,  $\text{active}(M) = \{i : M[i] = 1\}$ .

### Арифметические инструкции

`add.s32 %d, %a, %b:`

$$R'[t][\text{s32}][d] = (R[t][\text{s32}][a] + R[t][\text{s32}][b]) \bmod 2^{32}, \quad \forall t \in \text{active}(M).$$

`sub.u64 %d, %a, %b:`

$$R'[t][\text{u64}][d] = (R[t][\text{u64}][a] - R[t][\text{u64}][b]) \bmod 2^{64}.$$

`mul.lo.s32 %d, %a, %b:`

$$R'[t][\text{s32}][d] = (R[t][\text{s32}][a] \cdot R[t][\text{s32}][b]) \bmod 2^{32}.$$

`mul.hi.u32 %d, %a, %b:`

$$R'[t][\text{u32}][d] = \left\lfloor \frac{R[t][\text{u32}][a] \cdot R[t][\text{u32}][b]}{2^{32}} \right\rfloor.$$

`fma.rn.f32 %d, %a, %b, %c:`

$$R'[t][\text{f32}][d] = \text{round}_n(R[t][\text{f32}][a] \cdot R[t][\text{f32}][b] + R[t][\text{f32}][c]).$$

`rcp.rn.f32 %d, %a:`

$$R'[t][\text{f32}][d] = \text{round}_n\left(\frac{1}{R[t][\text{f32}][a]}\right).$$

### Инструкции сравнения

`setp.lt.s32 %p, %a, %b:`

$$P'[t] = (R[t][\text{s32}][a] < R[t][\text{s32}][b]), \quad \forall t \in \text{active}(M).$$

`setp.gt.f32 %p|%q, %a, %b:`

$$P'[t] = (R[t][\text{f32}][a] > R[t][\text{f32}][b]), \quad Q'[t] = \neg P'[t].$$

`sel.p.f32 %d, %a, %b, %p:`

$$R'[t][\text{f32}][d] = \begin{cases} R[t][\text{f32}][a] & \text{если } P[t] = 1, \\ R[t][\text{f32}][b] & \text{если } P[t] = 0. \end{cases}$$

## Инструкции памяти

`ld.global.f32 %d, [%a]:`

$$R'[t][f32][d] = Mem_{\text{global}}[R[t][u64][a]].$$

`st.shared.f32 [%a], %b:`

$$Mem'_{\text{shared}}[R[t][u64][a]] = R[t][f32][b], \quad \forall t \in \text{active}(M).$$

## Инструкции управления потоком

`bra.uni $L:`

$$\langle \text{bra.uni } \$L, \sigma \rangle \rightarrow \sigma[\text{PC} \leftarrow \text{offset}(\$L)].$$

`ret:`

$$\langle \text{ret}, \sigma \rangle \rightarrow \sigma[\text{PC} \leftarrow pc\_return, cur\_func \leftarrow caller\_func].$$

## С Трассировка PDOM\_Diverge на примере вложенной дивергенции

Рассмотрим фрагмент CUDA-кода с двумя вложенными условными ветвлениями:

Листинг 17: Пример вложенной дивергенции

```
// tid = threadIdx.x
if (tid < 16) {           // outer if: threads 0-15 / 16-31
    if (tid < 8) {        // inner if: threads 0-7 / 8-15
        A();
    } else {
        B();
    }
    C();
} else {
    D();
}
E(); // reconvergence point
```

Начальная маска:  $M_0 = 0xFFFFFFFF$ . Базовые блоки: bb\_outer\_if, bb\_inner\_if, bb\_A, bb\_B, bb\_C, bb\_D, bb\_E (реконвергенция).

Пусть  $PC_E = 100$  — PC метки E,  $PC_C = 50$  — PC метки C.

Трассировка подтверждает, что LIFO-порядок снятия фреймов восстанавливает маски в порядке «внутреннее-первым», затем «внешнее-первым», что обеспечивает корректное исполнение всех ветвей при произвольной глубине вложенности.

Таблица 13: Трассировка стека дивергенции

| Шаг | Текущий PC / инструкция                                                         | Стек $S$ (вершина справа)                                                                                                                                                                                                                                             |
|-----|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | <code>setp.lt + @%p bra \$outer</code>                                          | Внешняя дивергенция:<br>$M_{\text{taken}} = 0x0000FFFF$ , $M_{\text{not}} = 0xFFFF0000$<br>push конвергенции $\{100, 0xFFFFFFFF\}$ ,<br>push «не взята» $\{PC_D, 0xFFFF0000\}$<br>$S = [\{100, M_0, T\}, \{PC_D, M_{\text{not}}, F\}]$                                |
| 2   | Исполнение <code>bb_inner_if: @%p bra \$inner</code><br>Внутренняя дивергенция: | $M = 0x0000FFFF$ ;<br>$M_{\text{tk}} = 0x000000FF$ , $M_{\text{nt}} = 0x0000FF00$<br>push $\{PC_C, 0x0000FFFF, T\}$ , push $\{PC_B, 0x0000FF00, F\}$<br>$S = [\{100, M_0, T\}, \{PC_D, M_{\text{not\_outer}}, F\}, \{PC_C, 0x0000FFFF, T\}, \{PC_B, 0x0000FF00, F\}]$ |
| 3   | Исполнение <code>bb_A</code> ( $M = 0x000000FF$ )                               | $S$ без изменений                                                                                                                                                                                                                                                     |
| 4   | $PC = PC_B$ (CheckStack)                                                        | Снимаем $\{PC_B, 0x0000FF00, F\}$ :<br>$M \leftarrow 0x0000FF00$ , $S = [\{100, M_0, T\}, \{PC_D, M_{\text{not}}, F\}, \{PC_C, 0x0000FFFF, T\}]$                                                                                                                      |
| 5   | Исполнение <code>bb_B</code> ( $M = 0x0000FF00$ )                               | $S$ без изменений                                                                                                                                                                                                                                                     |
| 6   | $PC = PC_C$ (CheckStack)                                                        | Снимаем $\{PC_C, 0x0000FFFF, T\}$ :<br>$M \leftarrow 0x0000FFFF$ , $S = [\{100, M_0, T\}, \{PC_D, M_{\text{not}}, F\}]$                                                                                                                                               |
| 7   | Исполнение <code>bb_C</code> ( $M = 0x0000FFFF$ )                               | $S$ без изменений                                                                                                                                                                                                                                                     |
| 8   | $PC = PC_D$ (CheckStack)                                                        | Снимаем $\{PC_D, M_{\text{not}}, F\}$ :<br>$M \leftarrow 0xFFFF0000$ , $S = [\{100, M_0, T\}]$                                                                                                                                                                        |
| 9   | Исполнение <code>bb_D</code> ( $M = 0xFFFF0000$ )                               | $S$ без изменений                                                                                                                                                                                                                                                     |
| 10  | $PC = 100$ (CheckStack)                                                         | Снимаем $\{100, M_0, T\}$ :<br>$M \leftarrow 0xFFFFFFFF$ , $S = []$                                                                                                                                                                                                   |
| 11  | Исполнение <code>bb_E</code> ( $M = 0xFFFFFFFF$ )                               | Полная реконвергенция                                                                                                                                                                                                                                                 |

## D Формат файла `profiling.txt`: аннотированный пример

Файл `profiling.txt` имеет иерархическую структуру: «запуск ядра → блок → варп → записи инструкций». Ниже приведён аннотированный фрагмент для ядра `fft_custom`.

Листинг 18: Пример `profiling.txt` для `fft_custom`

```
# ===== Kernel launch =====
[LAUNCH] kernel=_Z10fft_kernelPcfi grid=(32,1,1) block=(32,1,1)

# ===== Block (0,0,0) =====
[BLOCK] block_id=(0,0,0)

# ===== Warp 0 =====
[WARP] warp_id=0

# --- Instruction records ---
# Format: timestamp_ns | pc | func | bb | instr | mask | metrics...
1234567890 | 42 | fft_kernel | $BB0_1 | ld.global.f32 | FFFFFFFF |
    branch_efficiency=1.000000
    global_mem_transactions=1
    global_coalescing=1.000000

1234567910 | 43 | fft_kernel | $BB0_1 | fma.rn.f32 | FFFFFFFF |
    branch_efficiency=1.000000

# --- Divergent instruction (bit-reversal stage) ---
1234568100 | 61 | fft_kernel | $BB0_3 | @%p0 bra $BB0_5 | 0000FFFF |
    branch_efficiency=0.500000

# Branch "taken" (threads 0-15)
1234568150 | 62 | fft_kernel | $BB0_5 | st.shared.f32 | 0000FFFF |
    branch_efficiency=0.500000
    bank_conflicts=0

# Reconvergence
1234568200 | 75 | fft_kernel | $BB0_7 | bar.sync | FFFFFFFF |
    branch_efficiency=1.000000

[END_WARP]
[END_BLOCK]
[END_LAUNCH]
```

Описание полей:

- `timestamp_ns` — монотонная временная метка в наносекундах.
- `pc` — монотонный РС инструкции (индекс в таблице).
- `func` — имя РТХ-функции.
- `bb` — имя базового блока.
- `instr` — мнемоника с предикатом (если есть).

- `mask` — маска исполнения в шестнадцатеричном формате.
- `metrics` — список пар `name=value`.

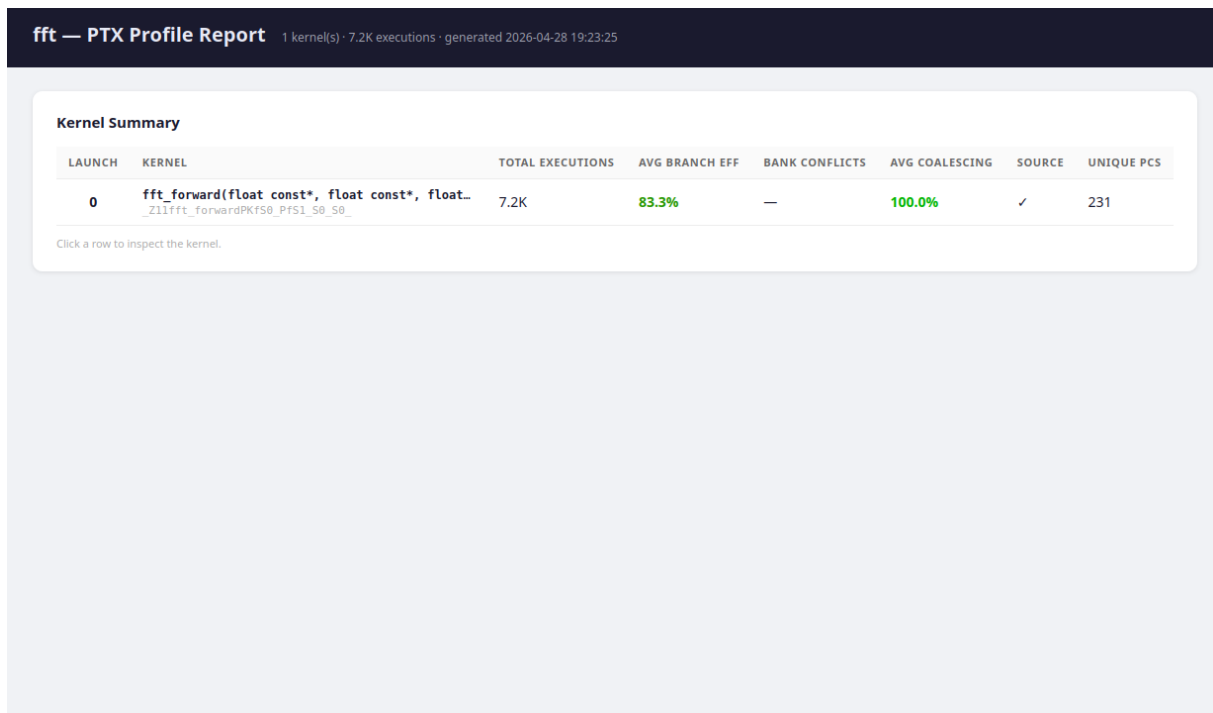
## Е Структура HTML-отчёта: описание секций

HTML-отчёт генерируется командой:

```
python -m tools.ptx_report \
  --profiling profiling.txt \
  --ptx fft_kernel.ptx \
  --output report_fft.html
```

Отчёт содержит пять секций:

Секция 1 — Панель «пилюль» (рисунок 4): сводные метрики ядра — общее число инструкций, число исполненных потоко-операций, среднее  $\bar{\eta}_{\text{branch}}$ , суммарные  $C_{\text{bank}}$ , средний  $\bar{\kappa}$ .



| LAUNCH | KERNEL                                                                               | TOTAL EXECUTIONS | AVG BRANCH EFF | BANK CONFLICTS | AVG COALESCING | SOURCE | UNIQUE PCS |
|--------|--------------------------------------------------------------------------------------|------------------|----------------|----------------|----------------|--------|------------|
| 0      | fft_forward(float const*, float const*, float...<br>_z1lfft_forwardPKfS0_PfS1_S0_S0_ | 7.2K             | 83.3%          | —              | 100.0%         | ✓      | 231        |

Click a row to inspect the kernel.

Рисунок 4: Панель «пилюль» HTML-отчёта.

Секция 2 — Таблица инструкций (рисунок 5): все уникальные РС с тепловой подсветкой по `exes_count`; столбцы: РС, мнемоника, блок, метрики.

Секция 3 — Горячие точки: топ-10 инструкций по дивергенции (минимальный  $\eta_{\text{branch}}$ ), по конфликтам банков (максимальный  $C_{\text{bank}}$ ), по слиянию (минимальный  $\kappa$ ).

Секция 4 — Таблица регистров: для каждого префикса регистров — тип, объявлено слотов, накопленный `write_ops`.

Секция 5 — Аннотированный исходный код: строки исходного `.cu`-файла с вложенными метриками инструкций, привязанных через `.loc`-директивы.

| PTX ASSEMBLY |                                              |            |            |           |            |
|--------------|----------------------------------------------|------------|------------|-----------|------------|
| PC           | PTX Instruction                              | Exec Count | Branch Eff | Mem Trans | Coalescing |
| 0x001e       | or.b32 %r20, %r19, %r16;                     | 32         | 100.0%     | —         | —          |
| fft_c_       |                                              |            |            |           |            |
| 0x001f       | or.b32 %r21, %r20, %r1;                      | 32         | 100.0%     | —         | —          |
| fft_c_       |                                              |            |            |           |            |
| 0x0020       | cvta.to.global.u64 %rd9, %rd5;               | 32         | 100.0%     | —         | —          |
| fft_c_       |                                              |            |            |           |            |
| 0x0021       | mul.wide.s32 %rd10, %r21, 4;                 | 32         | 100.0%     | —         | —          |
| 0x0022       | add.s64 %rd11, %rd9, %rd10;                  | 32         | 100.0%     | —         | —          |
| 0x0023       | ld.global.f32 %f56, [%rd11];                 | 32         | 100.0%     | 32        | 100.0%     |
| 0x0024       | mov.u32 %r22, _ZZ11fft_forwardPKfs0_Pfs1_... | 32         | 100.0%     | —         | —          |
| 0x0025       | add.s32 %r8, %r22, %r13;                     | 32         | 100.0%     | —         | —          |
| 0x0026       | st.shared.f32 [%r8], %f56;                   | 32         | 100.0%     | —         | —          |
| fft_c_       |                                              |            |            |           |            |
| 0x0027       | cvta.to.global.u64 %rd12, %rd6;              | 32         | 100.0%     | —         | —          |
| fft_c_       |                                              |            |            |           |            |
| 0x0028       | add.s64 %rd13, %rd12, %rd10;                 | 32         | 100.0%     | —         | —          |
| 0x0029       | ld.global.f32 %f57, [%rd13];                 | 32         | 100.0%     | 32        | 100.0%     |
| 0x002a       | mov.u32 %r23, _ZZ11fft_forwardPKfs0_Pfs1_... | 32         | 100.0%     | —         | —          |
| 0x002b       | add.s32 %r9, %r23, %r13;                     | 32         | 100.0%     | —         | —          |
| 0x002c       | st.shared.f32 [%r9], %f57;                   | 32         | 100.0%     | —         | —          |
| fft_c_       |                                              |            |            |           |            |
| 0x002d       | bar.sync 0;                                  | 32         | 100.0%     | —         | —          |
| fft_c_       |                                              |            |            |           |            |
| 0x002e       | xor.b32 %r24, %r13, 4;                       | 32         | 100.0%     | —         | —          |
| 0x002f       | add.s32 %r25, %r22, %r24;                    | 32         | 100.0%     | —         | —          |
| fft_c_       |                                              |            |            |           |            |
| 0x0030       | add.s32 %r26, %r23, %r24;                    | 32         | 100.0%     | —         | —          |
| fft_c_       |                                              |            |            |           |            |
| 0x0031       | ld.shared.f32 %f1, [%r25];                   | 32         | 100.0%     | —         | —          |

Рисунок 5: Таблица инструкций с тепловой подсветкой.

| SOURCE |                                             |     |      | PTX ASSEMBLY |                           |            |            |         |            |
|--------|---------------------------------------------|-----|------|--------------|---------------------------|------------|------------|---------|------------|
| Line   | Source                                      | PTX | Exec | PC           | PTX Instruction           | Exec Count | Branch Eff | Mem Tra | Coalescing |
| 36     | const int half = 1 << s;                    |     |      | fft_c_       |                           |            |            |         |            |
| 37     | const int partner = tid ^ half;             |     |      | 0x0038       | ld.shared.f32 %f6, [...]  | 32         | 100.0%     | —       | —          |
| 38     | const int j = tid & (half - 1);             |     |      | fft_c_       |                           |            |            |         |            |
| 39     | const int is_lower = (tid >> s) & 1;        |     |      | 0x0039       | ld.shared.f32 %f7, [...]  | 32         | 100.0%     | —       | —          |
| 40     |                                             |     |      | fft_c_       |                           |            |            |         |            |
| 41     | const float wr = tw_r[s * (FFT_N / 2) + j]; | 14  | 448  | 0x003a       | mul.f32 %f59, %f2, %...   | 32         | 100.0%     | —       | —          |
| 42     | const float wi = tw_i[s * (FFT_N / 2) + j]; | 9   | 288  | 0x003b       | fma.rm.f32 %f8, %f4, ...  | 32         | 100.0%     | —       | —          |
| 43     |                                             |     |      | fft_c_       |                           |            |            |         |            |
| 44     | const float a_r = sr[tid];                  | 5   | 160  | 0x003c       | bar.sync 0;               | 32         | 100.0%     | —       | —          |
| 45     | const float a_i = si[tid];                  | 5   | 160  | fft_c_       |                           |            |            |         |            |
| 46     | const float b_r = sr[partner];              | 15  | 480  | 0x003d       | setp.eq.s32 %p1, %r3, ... | 32         | 100.0%     | —       | —          |
| 47     | const float b_i = si[partner];              | 10  | 320  | 0x003e       | @p1 bra \$L_BB0_2;        | 32         | 50.0%      | —       | —          |
| 48     |                                             |     |      | 0x003f       | bra.uni \$L_BB0_1;        | 32         | 50.0%      | —       | —          |
| 49     | const float tb_r = wr * b_r - wi * b_i;     | 15  | 480  | fft_c_       |                           |            |            |         |            |
| 50     | const float tb_i = wr * b_i + wi * b_r;     | 10  | 320  | 0x0040       | mul.f32 %f64, %f4, %...   | 32         | 50.0%      | —       | —          |
| 51     | const float ta_r = wr * a_r - wi * a_i;     | 15  | 480  | 0x0041       | mul.f32 %f65, %f2, %...   | 32         | 50.0%      | —       | —          |
| 52     | const float ta_i = wr * a_i + wi * a_r;     | 10  | 320  | 0x0042       | sub.f32 %f66, %f65, ...   | 32         | 50.0%      | —       | —          |
| 53     |                                             |     |      | fft_c_       |                           |            |            |         |            |
| 54     | __syncthreads();                            | 5   | 160  | 0x0043       | add.f32 %f67, %f6, %...   | 32         | 50.0%      | —       | —          |
| 55     |                                             |     |      | 0x0044       | st.shared.f32 [%r8], ...  | 32         | 50.0%      | —       | —          |
| 56     | if (!is_lower)                              | 15  | 480  | fft_c_       |                           |            |            |         |            |
| 57     | {                                           |     |      | 0x0045       | add.f32 %f110, %f7, ...   | 32         | 50.0%      | —       | —          |
| 58     | sr[tid] = a_r + tb_r;                       | 10  | 320  | 0x0046       | bra.uni \$L_BB0_3;        | 32         | 50.0%      | —       | —          |
| 59     | si[tid] = a_i + tb_i;                       | 10  | 320  | fft_c_       |                           |            |            |         |            |
| 60     | }                                           |     |      | 0x0047       | mul.f32 %f60, %f4, %...   | 32         | 50.0%      | —       | —          |
| 61     | else                                        |     |      | 0x0048       | mul.f32 %f61, %f2, %...   | 32         | 50.0%      | —       | —          |
| 62     | {                                           |     |      | 0x0049       | sub.f32 %f62, %f61, ...   | 32         | 50.0%      | —       | —          |
| 63     | sr[tid] = b_r - ta_r;                       | 10  | 320  | fft_c_       |                           |            |            |         |            |
| 64     | si[tid] = b_i - ta_i;                       | 5   | 160  | 0x004a       | sub.f32 %f63, %f1, %...   | 32         | 50.0%      | —       | —          |

Рисунок 6: Аннотированный исходный код в HTML-отчёте.